



Fourier warm start for physics-informed neural networks

Ge Jin^{a,b}, Jian Cheng Wong^{b,c}, Abhishek Gupta^d, Shipeng Li^{a,*}, Yew-Soon Ong^{b,c}

^a School of Aerospace Engineering, Beijing Institute of Technology, Beijing, 100081, China

^b School of Computer Science and Engineering, Nanyang Technological University, 639798, Singapore

^c Agency for Science, Technology and Research (A*STAR), 138632, Singapore

^d School of Mechanical Sciences, Indian Institute of Technology (IIT) Goa, Ponda, 403401, Goa, India

ARTICLE INFO

Keywords:

Fourier warm start
Physics-informed neural networks
Spectral bias
Neural tangent kernel
Multi-frequency

ABSTRACT

Physics-informed neural networks (PINNs) have shown applicability in a wide range of engineering domains. However, there remain some challenges in their use, namely, PINNs are notoriously difficult to train and prone to failure when dealing with complex tasks with multi-frequency patterns or steep gradients in the outputs. In this work, we leverage the Neural Tangent Kernel (NTK) theory and introduce the Fourier Warm Start (FWS) algorithm to balance the convergence rate of neural networks at different frequencies, thereby mitigating spectral bias and improving overall model performance. We then propose the Fourier Analysis Boosted Physics-Informed Neural Network (*Fab*-PINN), a novel integrated architecture based on the FWS algorithm. Finally, we present a series of challenging numerical examples with multi-frequency or sparse observations to validate the effectiveness of the proposed method. Compared to standard PINN, *Fab*-PINN exhibits a reduction of relative L2 errors in solving the heat transfer equation, the Klein–Gordon equation, and the transient Navier–Stokes equations from 9.9×10^{-1} to 4.4×10^{-3} , 5.4×10^{-1} to 2.6×10^{-3} , and 6.5×10^{-1} to 9.6×10^{-4} , respectively.

1. Introduction

Due to the universal approximation capacity of neural networks, solving difficult mathematical physics equations with deep learning methods is becoming a promising research area. One of the most popular and emerging approaches is the physics-informed neural network (PINN) (Raissi et al., 2019, 2020), which is considered a viable alternative to conventional numerical solvers and has achieved notable results in many disciplines such as biology, geology, hydraulics, materials, electromagnetics, or atmospheric sciences (Cuomo et al., 2022; Cai et al., 2021). Despite the early successes, PINN models sometimes struggle to achieve convergence or may fail entirely on complex problems (Krishnapriyan et al., 2021; Wang et al., 2021a), particularly those involving multiple frequencies or high frequencies (Wang et al., 2022; Song and Wang, 2023; Sallam and Fürth, 2023), which are, however, ubiquitous in engineering scenarios. As PINNs have received widespread attention and are beginning to penetrate into practical applications, it is becoming increasingly urgent to address the training pathologies of PINNs to further enhance their performance.

In data-driven tasks, there is a known deficiency of neural networks (NNs) in learning multi-frequency or high-frequency functions, referred to as *spectral bias* (Rahaman et al., 2019; Xu et al., 2019; Cao et al., 2021; Geifman et al., 2022). It is considered an inherent property of NNs, and it is much more severe for PINNs due to the

additional constraints. Recently, several approaches have been proposed to alleviate this problem, such as PhaseDNN (Cai et al., 2020), MscaleDNN (Liu et al., 2020; Wang et al., 2020), and curriculum regularization method (Krishnapriyan et al., 2021). Moreover, by incorporating the traditional domain decomposition method (Smith, 1997), the ability of PINNs to handle the high frequency and complex domain problems can also be enhanced (Li et al., 2019; Jagtap et al., 2020; Kharazmi et al., 2021).

The above approaches can mitigate the spectral bias at the cost of a significant increase in computational or algorithmic complexity. The following methods, however, give us a glimpse of a radical and straightforward solution to the spectral bias. After Rahimi and Recht (2007a) first proposed the random Fourier feature method to accelerate the training of kernel machines, Tancik et al. (2020a) integrate it with fully-connected networks and show experimentally that tuning the Fourier feature parameters to an appropriate value can significantly improve the high-frequency learning performance and overcome the spectral bias. The Fourier feature method has therefore also been introduced to physics-informed learning, achieving similar results (Lu et al., 2021; Wang et al., 2021b). Developments have further combined the method with a multiscale architecture and domain decomposition (Li et al., 2023). Moreover, with the recent Neural Tangent Kernel (NTK) theory (Jacot et al., 2018), the training dynamics of such neural networks

* Corresponding author.

E-mail address: isp@bit.edu.cn (S. Li).

Table 1
List of abbreviations.

Acronym	Full term(s)
AD	Automatic Differentiation
CF	Cosinusoidal Feature
DFT	Discrete Fourier Transform
DL	Deep Learning
<i>Fab</i> -PINN	Fourier Analysis Boosted Physics-Informed Neural Network
FF	Fourier Feature
FWS	Fourier Warm Start
HL	Hidden Layer
IL	Identity Layer
L-BFGS	Limited memory Broyden–Fletcher–Goldfarb–Shanno
ML	Mapping Layer
MLP	Multi-Layer Perceptrons
MSE	Mean Squared Error
NAW	Normalized Adaptive Weights
NN	Neural Network
NTK	Neural Tangent Kernel
ODE	Ordinary Differential Equation
PDE	Partial Differential Equation
PINN	Physics-Informed Neural Network
PIV	Particle Image Velocimetry
RFF	Random Fourier Feature
SF	Sinusoidal Feature
SGD	Stochastic Gradient Descent

can be rigorously analyzed and elucidated. It has been shown that a fully-connected neural network in the infinite-width limit behaves like a kernel regression model with a deterministic kernel, which is the so-called NTK. Further studies on NTK show that the training convergence rates of the different components of the target solution are directly determined by the corresponding eigenvalues of the NTK (Lee et al., 2019; Ronen et al., 2019; Basri et al., 2020). In particular, the Fourier feature method can be used to modulate the NTK eigenvalues of a neural network, thus controlling the convergence rate of different components (Tancik et al., 2020a; Wang et al., 2021b).

Motivated by the foregoing developments, this work analyzes the relationship between the convergence rate for different frequencies and the NTK eigenvalues, with a particular focus on the effect of the initialization of the Fourier feature parameters. We find a new solution to the spectral bias problem and propose a novel unified architecture to further improve the performance of PINNs.

Our contributions. The specific contributions of this paper can be summarized as follows:

- We find that for the Fourier feature neural network, the frequency of the NTK eigenfunctions and the relevant NTK eigenvalues can be manipulated by simultaneously adjusting the Fourier feature parameters and the number of corresponding neurons, thus modifying the convergence rate of the neural network at the specified frequency.
- We leverage the aforementioned insights to present a Fourier Warm Start method that allows the convergence rate of the neural network to be manipulated for different frequencies to mitigate the spectral bias, resulting in a smoother training loss landscape and better model performance.
- We propose a novel architecture called *Fab*-PINN that provides a unified strategy to address the training challenges of PINNs. It integrates the Fourier Warm Start to mitigate spectral bias and an identity layer to preserve the Fourier feature mapping.
- We experiment on several challenging benchmark problems, including solving the heat transfer equation, multi-frequency Klein–Gordon equation, and transient Navier–Stokes equations with multi-frequency or sparse observations. The proposed method shows significant performance improvements over previous methods.

Table 1 presents the most used acronyms in this paper. The rest of the paper is organized as follows. In Section 2, we present a brief overview of PINNs and introduce some preliminaries, including the discrete Fourier transform, the Fourier feature mapping method, and the NTK theory, which are the basis of our proposed approaches. Next, in Section 3, we first introduce an approach to manipulate the NTK eigenvalues and the frequency of the NTK eigenfunctions of the Fourier feature neural network. Then, based on the analysis, we introduce the FWS method. After that, we employ the Fourier Warm Start (FWS) method for physics-informed tasks and propose a novel integrated architecture called *Fab*-PINN. Finally, in Section 4, we perform a series of numerical experiments to further validate the performance and effectiveness of the proposed method.

2. Preliminaries

2.1. Physics-informed neural networks

Physics-informed neural networks (PINNs) are a class of deep learning algorithms that seamlessly integrate data and mathematical physics models, and have shown tremendous potential to become a unified deep learning (DL) based PDE solver (Raissi et al., 2019, 2020). In a wide range of scientific and engineering scenarios, the mathematical physics models can be described as partial differential equations (PDEs). The general form of a PDE can be abstracted as follows:

$$\mathcal{P}(u(x, t)) = f(x, t), \quad x \in \Omega \subset \mathbb{R}^d, \quad t \in [0, T], \quad (1a)$$

$$u(x, 0) = h(x), \quad x \in \Omega, \quad (1b)$$

$$\mathcal{B}(u(x, t)) = g(x, t), \quad x \in \partial\Omega, \quad t \in [0, T], \quad (1c)$$

where $u(x, t)$ is the state variable, x and t are the space and time coordinates, Ω and T are the space domain and the time horizon. $\mathcal{P}(\cdot)$ and $\mathcal{B}(\cdot)$ are the partial differential operator and the boundary operator. $f(x, t)$, $h(x)$ and $g(x, t)$ are the source function, initial condition, and boundary condition, respectively. We use u_θ as the neural network prediction to approximate the target solution $u(x, t)$. The parameters θ (i.e. the weights and biases of the neural network) are found by minimizing the composite loss function of the general form

$$\mathcal{L}(\theta) = \lambda_l \mathcal{L}_l(\theta) + \lambda_r \mathcal{L}_r(\theta), \quad (2)$$

where $\mathcal{L}_l(\theta)$ is the labeled loss, which constrains the prediction results to label points, and $\mathcal{L}_r(\theta)$ is the residual loss, which penalizes the constraint PDE to sampled residual points. The parameters λ_l and λ_r correspond to weighting coefficients in the loss function that control the tradeoff between different components of the composite loss function. In addition, the r.h.s. of equation Eq. (2) can be written in different forms depending on the specific type of problem being investigated. For example, if a forward dynamic problem is studied, the composite loss can be expressed as the sum of several squared norm terms L^2 with respect to the initial condition constraint, the boundary condition constraint, and the PDE residual constraint,

$$\begin{aligned} \mathcal{L}(\theta) &= \lambda_{l1} \|u_\theta|_{t=0} - h\|_{L^2(\Omega)}^2 + \lambda_{l2} \|B(u_\theta) - g\|_{L^2(T; \partial\Omega)}^2 \\ &\quad + \lambda_r \|P(u_\theta) - f\|_{L^2(T; \Omega)}^2 \\ &= \lambda_{l1} \int_{\Omega} |u_\theta(x, 0) - h(x)|^2 dx \\ &\quad + \lambda_{l2} \int_0^T \int_{\partial\Omega} |B(u_\theta(x, t)) - g(x, t)|^2 dx dt \\ &\quad + \lambda_r \int_0^T \int_{\Omega} |P(u_\theta(x, t)) - f(x, t)|^2 dx dt, \end{aligned} \quad (3)$$

where λ_{l1} , λ_{l2} , and λ_r represent the weighting coefficients for initial condition, boundary condition and PDE residuals, respectively. When considering an inverse problem, the labeled loss typically consists only

of observation constraints in the computational domain. The composite loss can be formulated as

$$\begin{aligned}\mathcal{L}(\theta) &= \lambda_l \left\| u_\theta(x, t) - u(x, t) \right\|_{L^2(T; \Omega)}^2 + \lambda_r \left\| P(u_\theta) - f \right\|_{L^2(T; \Omega)}^2 \\ &= \lambda_l \int_0^T \int_\Omega |u_\theta(x, t) - u(x, t)|^2 dx dt \\ &\quad + \lambda_r \int_0^T \int_\Omega |P(u_\theta(x, t) - f(x, t))|^2 dx dt,\end{aligned}\quad (4)$$

where λ_l here represents the weighting coefficient for labeled data. In practice, the integrals in Eqs. (3) and (4) are typically approximated by discrete collocation points sampled in the corresponding computational domains. The derivatives in the operator can be computed directly by the Automatic Differentiation (AD) method (Baydin et al., 2018). The process of minimizing the composite loss function, also known as finding the optimal parameters, can be implemented by typical optimization methods, including Stochastic Gradient Descent (SGD) (Bottou et al., 2018), Limited memory Broyden–Fletcher–Goldfarb–Shanno (L-BFGS) algorithm (Saputro and Widyaningsih, 2017), or Neuroevolution Algorithms (Wong et al., 2021).

2.2. Fourier feature neural networks and variants

A Fourier feature neural network maps the input coordinates to a feature space with Fourier feature mapping before passing them to hidden layers. Fourier feature mapping was first introduced as random Fourier features approximating an arbitrary stationary kernel function to speed up the training of kernel machines (Rahimi and Recht, 2007b). Subsequently, the method has yielded significant performance improvements in combination with multi-layer perceptrons (MLPs) (Tancik et al., 2020b) and physics-informed neural networks (PINNs) (Wang et al., 2021b; Li et al., 2023). The variants of Fourier feature mapping, such as sinusoidal feature mapping (Wong et al., 2022), have also shown tremendous application potential. We briefly introduce Fourier Feature Mapping and its variants as follows:

Random Fourier Feature (RFF) Mapping. The RFF mapping considers the approximation of an arbitrary stationary kernel function that featurize the input spatiotemporal coordinates $\mathbf{v} \in \mathbb{R}^d$ to the surface of a higher dimensional hypersphere. The mapping function can be expressed as:

$$\gamma(\mathbf{v}) = [\cos(2\pi \mathbf{B}^* \mathbf{v}), \sin(2\pi \mathbf{B}^* \mathbf{v})]^T \quad (5)$$

where the Fourier base frequency $\mathbf{B}^* \in \mathbb{R}^{m \times d}$ is randomly sampled from Gaussian distribution $\mathcal{N}(0, \sigma^2)$. σ is a hyperparameter that determines the scale of the distribution of the Fourier base frequencies and must be tuned based on the specific task and data set. m is the number of basis functions in the Fourier basis set. In RFF mapping, once $\mathbf{B}^*$ is selected, it remains frozen in the subsequent neural network training process.

Fourier Feature (FF) Mapping. Here by default, FF mapping stands for trainable FF mapping to distinguish it from RFF mapping, and it has the following expression:

$$\gamma(\mathbf{v}) = [\cos(2\pi \mathbf{B} \mathbf{v} + \mathbf{b}), \sin(2\pi \mathbf{B} \mathbf{v} + \mathbf{b})]^T \quad (6)$$

where $\mathbf{B} \in \mathbb{R}^{m \times d}$ and $\mathbf{b} \in \mathbb{R}^{m \times d}$ are trainable parameters and can be considered as part of the weights and biases of the neural network. The initialization of $\mathbf{B}$ and $\mathbf{b}$ is $\mathcal{N}(0, \sigma^2)$ and zero, where σ must be set by hyperparameter sweep or relying on experience. The default value of σ is set to 1 in this paper.

Sinusoidal Feature (SF) Mapping. The SF mapping removes the cosine function from the mapping function since the cosine function can be transformed into a sine function by a fixed phase shift. It can be represented as:

$$\gamma(\mathbf{v}) = [\sin(2\pi (\mathbf{B} \mathbf{v} + \mathbf{b}))]^T \quad (7)$$

Cosinusoidal Feature (CF) Mapping. As mentioned above, the cosine function differs from the sine function by a constant phase

difference, and this phase difference can be accommodated in the trainable phase parameter $\mathbf{b}$. Thus, the CF mapping can be derived as:

$$\gamma(\mathbf{v}) = [\cos(2\pi (\mathbf{B} \mathbf{v} + \mathbf{b}))]^T \quad (8)$$

In SF and CF mapping, the definition and initialization of $\mathbf{B}$ and $\mathbf{b}$ are the same as in FF mapping.

2.3. Neural Tangent Kernel Theory

The neural tangent kernel theory (Jacot et al., 2018; Lee et al., 2019) links neural networks with Gaussian processes and proves that the evolution of a network during training in the infinite-width limit can be described by a new kernel, the so-called Neural Tangent Kernel (NTK). NTK can be employed as a powerful and effective tool to study the training dynamics of deep learning models.

NTK for Fully-Connected Neural Network. For a fully-connected infinite-width neural network, when it is trained by a gradient descent algorithm with a very small learning rate η on the mean squared error (MSE), it behaves like a kernel regression model with a deterministic kernel called Neural Tangent Kernel (Jacot et al., 2018; Lee et al., 2019). The entries of NTK can be expressed as:

$$\mathbf{K}_{ij}(t) = \mathbb{E}_{\theta \sim \mathcal{N}} \left\langle \frac{\partial \mathbf{u}(\mathbf{v}_i; \theta(t))}{\partial \theta}, \frac{\partial \mathbf{u}(\mathbf{v}_j; \theta(t))}{\partial \theta} \right\rangle, \quad (9)$$

where $\mathbf{u}(\mathbf{v}; \theta)$ is the output of the network and θ are network parameters including weights $\mathbf{W}$ and biases $\mathbf{b}$. Considering that the output of the NN after initialization is close to zero, the solution of the first-order Taylor expansion can be calculated according to the asymptotic condition, and the training error $\mathbf{u}(\mathbf{v}_{\text{train}}; \theta(t)) - \mathbf{u}_{\text{train}}$ at iteration t can be obtained as follows (Novak et al., 2020; Tancik et al., 2020a):

$$\mathbf{Q}^T (\mathbf{u}(\mathbf{v}_{\text{train}}; \theta(t)) - \mathbf{u}_{\text{train}}) \approx -e^{-\eta \Lambda t} \mathbf{Q}^T \mathbf{u}_{\text{train}}, \quad (10)$$

where $\mathbf{Q}$ and Λ are the orthogonal and diagonal matrix from the eigendecomposition of the NTK matrix $\mathbf{K} = \mathbf{Q} \Lambda \mathbf{Q}^T$. From this relationship, we can see that the i th eigenvalues λ_i in the diagonal matrix Λ can determine the decay rate of the corresponding training error component.

NTK for Physics-Informed Neural Network. For a physics-informed neural network, consider minimizing the loss function described in Eq. (2) by gradient descent with an infinitesimally small learning rate. Following the derivation of Wang et al. (2021b, 2022), we can express the NTK of PINN as follows

$$\mathbf{K}(t) = \begin{bmatrix} \mathbf{K}_{uu}(t) & \mathbf{K}_{ur}(t) \\ \mathbf{K}_{ru}(t) & \mathbf{K}_{rr}(t) \end{bmatrix}. \quad (11)$$

Similar to Eq. (10), approximating the NTK linear system model by the first-order Taylor expansion around the initial parameters, the training error matrix of PINN can be derived as (Wang et al., 2022)

$$\mathbf{Q}^T \begin{bmatrix} \mathbf{u}(\mathbf{v}_{\text{train}}; \theta(t)) - \mathbf{u}_{\text{train}} \\ \mathcal{P}[u](\mathbf{v}_r; \theta(t)) - f_r \end{bmatrix} \approx e^{-\eta \Lambda t} \mathbf{Q}^T \begin{bmatrix} \mathbf{u}(\mathbf{v}_{\text{train}}; \theta(0)) - \mathbf{u}_{\text{train}} \\ \mathcal{P}[u](\mathbf{v}_r; \theta(0)) - f_r \end{bmatrix}. \quad (12)$$

For the NTK matrix of PINN, the sum of the eigenvalues λ_i in the diagonal matrix Λ can be expressed as

$$\sum_{i=1}^{N_l} \lambda_i^{uu}(t) + \sum_{i=1}^{N_r} \lambda_i^{rr}(t) = \text{Tr}(\mathbf{K}_{uu}(t)) + \text{Tr}(\mathbf{K}_{rr}(t)), \quad (13)$$

where $\text{Tr}(\cdot)$ means the trace of the matrix.

2.4. Short review of the Discrete Fourier Transform (DFT)

The Fourier transform (FT), a fundamental mathematical approach that transforms a function into a form that describes the frequency of the original function, is ubiquitous in a wide range of disciplines, including signal processing, communications, and control (Sneddon,

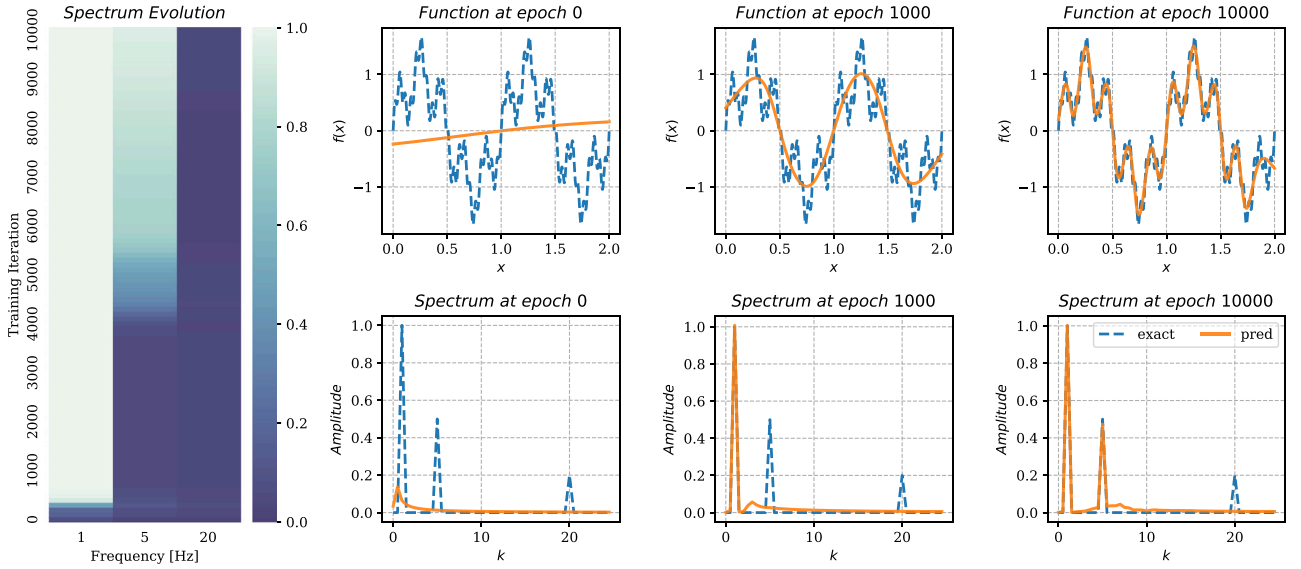


Fig. 1. Spectral bias in the training process. (Left) The evolution of the spectrum for different frequencies during training. The lighter color indicates that the spectrum at the corresponding frequency is closer to the target amplitude. (Right) The network predictions and their frequency distributions at epoch 0, 1k, and 10k, respectively. Detailed training settings can be found in Section 3.2.2. **Gist:** The model prioritizes learning lower frequencies first, and higher frequencies require more training time (iterations).

1995). In engineering practice, the Discrete Fourier Transform (DFT) is usually adopted for the convenience of numerical calculations. The DFT, which is a discrete approximation of the Fourier series, is suitable for periodic functions. However, even for a normal function f measured at $N \in \mathbb{N}$ discretely sampled points over a bounded computational domain, it can be regarded as a periodic function by adding imaginary samples outside the domain of the actual points. These imaginary samples duplicate the measurements at the actual points, inducing periodicity. The DFT pair $\hat{F}$ and $\hat{F}^{-1}$ can be written as

$$F(\xi_c) = \hat{F}(f(x_n)) = \sum_{n=0}^{N-1} f(x_n) e^{-i \frac{2\pi}{N} cn}, \quad c = 0, \dots, N-1 \quad (14a)$$

$$f(x_n) = \hat{F}^{-1}(F(\xi_c)) = \frac{1}{N} \sum_{c=0}^{N-1} F(\xi_c) e^{i \frac{2\pi}{N} cn}, \quad n = 0, \dots, N-1 \quad (14b)$$

where ξ_c is frequency, $i = \sqrt{-1}$ is the imaginary unit, and $e^{i\theta} = \cos \theta + i \sin \theta$.

In engineering applications such as signal processing or image processing, it is common practice to extract the main or important frequencies by performing a DFT on the original data, followed by intercepting a limited number of low frequencies and performing the inverse transform. This approach is commonly known as frequency domain filtering, which allows for the manipulation of specific frequency components of the signal or image (Orfanidis, 1995; Rao et al., 2010).

3. Proposed methodology and theoretical analysis

Recent research has found that neural networks tend to prioritize learning low frequencies when learning the target function (Rahaman et al., 2019; Ronen et al., 2019). This phenomenon is known as spectral bias and can be observed clearly in Fig. 1. The spectral bias leads to an imbalance in the convergence rate during the neural network training process, severely limiting the overall performance of the model. Such inherent properties of neural networks also cause training pathologies that severely affect the performance of PINNs, especially when dealing with multi-frequency and other intricate problems. Therefore, in this section, we will start from the spectral bias phenomenon and thereafter propose new methods, algorithms, and novel architectures to improve the performance of PINNs.

3.1. Manipulating the spectral bias by fourier feature mapping

We define a neural network $Q : \mathbb{R}^d \mapsto \mathbb{R}$ with L hidden layers of width $d_1, \dots, d_L$ and a single output neuron:

$$Q(\mathbf{v}; \mathbf{W}, \mathbf{b}) := \mathcal{L}_{L+1} \circ \phi \mathcal{L}_L \circ \dots \circ \phi \mathcal{L}_1(\mathbf{v}), \quad (15)$$

$$\mathcal{L}_l(\cdot) = \mathbf{W}_l \times (\cdot) + \mathbf{b}_l, \quad l = 1, \dots, L+1,$$

where $\mathbf{v} \in \mathbb{R}^d$ is the input spatio-temporal coordinate vector with a dimension d , and $\mathcal{L}$ represents a linear layer with parameters including weights $\mathbf{W}$ and biases $\mathbf{b}$. ϕ represents the fixed activation function. The output of Q is defined as $\mathbf{u} \in \mathbb{R}$.

From the relationship described in Eq. (10), the decay rate of the training error component along the eigenfunctions is determined by the corresponding eigenvalues. Therefore, the network will always prefer to approximate the target function along the eigenfunctions with larger eigenvalues. In particular, for a fully-connected neural network, the leading eigenvalues of the NTK usually correspond to low-frequency eigenfunctions, and the eigenvalues shrink considerably as the frequency of the eigenfunctions increases. This can be clearly seen in Fig. 2, where we plot four leading and four trailing eigenfunctions corresponding to the $\tanh$ activation function. We can also see that the eigenvalues decay greatly for high-frequency eigenfunctions. These results essentially reveal the bias of conventional neural networks towards learning lower frequencies.

Next, we introduce the special frequency response characteristics of the Fourier feature neural network, which embeds a Fourier feature (FF) mapping layer $\gamma(\mathbf{v})$ described in Eq. (6) before the hidden layers of the neural network. According to Tancik et al. (2020b), after embedding the Fourier feature mapping, the NTK of the model can be expressed as:

$$h_{\text{NTK}}(\gamma(\mathbf{v}_i)^T \gamma(\mathbf{v}_j)) = h_{\text{NTK}} \left(\sum_{k=1}^m a_k^2 \cos(2\pi \mathbf{B}_k^T (\mathbf{v}_i - \mathbf{v}_j)) \right). \quad (16)$$

We can see that the Fourier feature mapping allows the NTK function to be stationary and allows us to manipulate the eigenvalues and corresponding eigenfunctions of the NTK matrix. To further reveal the effect of the Fourier feature parameters on the NTK eigenvalues and eigenfunctions, we consider a special two-layer (one-hidden layer) bias-free neural network with one-dimensional input, and its first layer is a

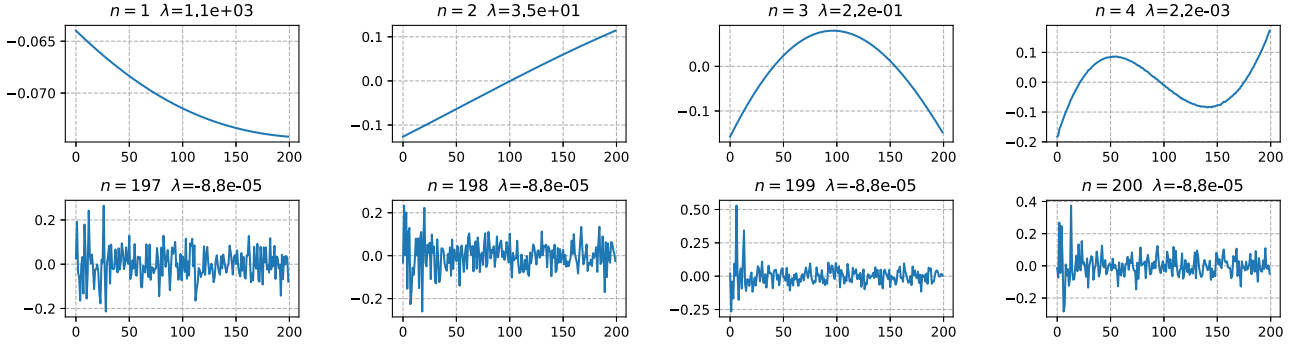


Fig. 2. Eigenvalues and corresponding eigenfunctions. Four leading and four trailing eigenvalues with corresponding eigenfunctions in descending order. **Gist:** For conventional neural networks, larger eigenvalues usually correspond to lower-frequency eigenfunctions, and the eigenvalues shrink considerably as the frequency of the eigenfunctions increases.

simplified Fourier feature mapping layer,

$$\mathcal{Q}_{FF}^*(\mathbf{v}; \mathbf{W}) = \mathbf{W} \cdot \begin{bmatrix} \cos(2\pi \mathbf{B}^T \mathbf{v}) \\ \sin(2\pi \mathbf{B}^T \mathbf{v}) \end{bmatrix}, \quad (17)$$

note that $\mathbf{B} = [B_1, B_2, \dots, B_N] \in \mathbb{R}^N$ is the Fourier feature parameter which is initialized as a Gaussian distribution $\mathcal{N}(\mathbf{B}, \epsilon^2)$, ϵ represents a constant small value. $\mathbf{v} \in \mathbb{R}$ is still the input coordinate vector and the computational domain is $\mathcal{V} = [0, 1)$. $\mathbf{W} \in \mathbb{R}^{1 \times 2N}$ is the weight matrix. The neural tangent kernel $\mathbf{K}$ of this model is given as:

$$\begin{aligned} \mathbf{K}_{ij} &= \mathbf{K}(\mathbf{v}_i, \mathbf{v}_j) = \begin{bmatrix} \cos(2\pi \mathbf{B}^T \mathbf{v}_i) \\ \sin(2\pi \mathbf{B}^T \mathbf{v}_i) \end{bmatrix}^T \cdot \begin{bmatrix} \cos(2\pi \mathbf{B}^T \mathbf{v}_j) \\ \sin(2\pi \mathbf{B}^T \mathbf{v}_j) \end{bmatrix} \\ &= \sum_{k=1}^N \cos(2\pi B_k \mathbf{v}_i) \cos(2\pi B_k \mathbf{v}_j) + \sin(2\pi B_k \mathbf{v}_i) \sin(2\pi B_k \mathbf{v}_j) \\ &= \sum_{k=1}^N \cos(2\pi B_k (\mathbf{v}_i - \mathbf{v}_j)) \\ &\approx N \cos(2\pi B (\mathbf{v}_i - \mathbf{v}_j)). \end{aligned} \quad (18)$$

From this simplified kernel function, the eigenfunctions are given by

$$g(\mathbf{v}) = C_1 \cos(2\pi B \mathbf{v}) + C_2 \sin(2\pi B \mathbf{v}), \quad (19)$$

where C_1 and C_2 are constants. The corresponding non-zero eigenvalues are

$$\lambda = N \left(\frac{1}{2} \pm \frac{\sin(2\pi B)}{4\pi B} \right), \quad (20)$$

as derived in [Appendix C](#), where we also have studied the variants of the FF neural network introduced in Section 2.2 and obtained similar results. We can find that in this simplified FF neural network, the frequency of the eigenfunctions can be directly specified by the parameter B , and the eigenvalues can be linearly adjusted by the corresponding number of neurons N .¹ Note that from Eq. (10) we learn that the magnitude of the eigenvalue determines the convergence rate of the corresponding component. This gives us a technique to *directly adjust the convergence rate of the neural network at frequency B by adjusting the number of corresponding neurons N* . We will now verify this conclusion through numerical experiments.

¹ From Eq. (20) we can find that, the parameter B and the number of neurons N both have effect on the NTK eigenvalues λ . However, the second term of the RHS of the Eq. (20) approaches zero as B becomes large. In this case, λ is only affected by N . Since in this work, we focus on the convergence rate of neural networks at higher frequencies, i.e., where the parameter B is typically much large than 1. When B is equal to 5, the effect of the second term on the whole equation is less than 5%, and when B is equal to 10, the effect has been reduced to about 1%. Therefore, we omit the study of the effect of the parameter B on NTK eigenvalues λ in the following numerical experiments.

Table 2

Leading eigenvalues for different numbers of neurons. The results are averaged from 5 independent trials.

Neuron number N	60	80	100	120
Leading eigenvalue λ	3004.3	4004.7	5004.4	6004.5

We consider a two-layer Fourier feature neural network with a width of N . To illustrate the effect of the initialization value of the Fourier feature parameter $\mathbf{B}$ and the number of initialization neurons N , we first consider four cases where B are 5, 10, 20, and 40, respectively. This means the Fourier feature parameters are randomly initialized from $\mathcal{N}(5, \epsilon^2)$, $\mathcal{N}(10, \epsilon^2)$, $\mathcal{N}(20, \epsilon^2)$, and $\mathcal{N}(40, \epsilon^2)$, respectively. In all cases, we choose $\epsilon = 10^{-5}$ and the number of corresponding neurons is kept as $N = 60$. We can find the leading eigenvalues and the leading eigenfunctions with their frequency distribution for these four cases in [Fig. 3](#). The results are aggregated from 5 independent trials.

In all cases, the principal frequency of the leading eigenfunction is naturally close to the specified frequency B . This means that after our manipulation the neural network prioritizes learning the specified frequency over other low frequencies. Moreover, the leading eigenvalues are hardly affected by the values of the Fourier feature parameters.

Next, we test the effect of the number of initialization neurons N . We consider four cases where the corresponding N are 60, 80, 100, and 120, respectively. In all cases, the specified initialization frequency is $B = 20$, and we still choose $\epsilon = 10^{-5}$ to initialize the Fourier feature parameters. The corresponding relationship of N with NTK leading eigenvalue can be found in [Table 2](#). The results are averaged from 5 independent trials. We can find that the proportion of the leading eigenvalues is about 3:4:5:6, identical to the proportion of the specified neural numbers. This experimental result substantiates our conclusion, i.e., *for FF neural networks, the frequency of the NTK eigenfunctions and the relevant NTK eigenvalues can be manipulated by setting the value of the Fourier feature parameters and the number of corresponding neurons, thus adjusting the convergence rate of the neural network at the specified frequency*.

Based on the above analysis, in the next section, we will further propose a method to automatically adjust the NTK of the neural network for different target functions, so that the neural network has a balanced convergence rate at all target frequencies and eliminates the spectral bias pathology.

3.2. Fourier warm start for spectral equilibrium

The bias by which the neural network preferentially learns lower frequencies is known as *spectral bias*, and here we refer to the state where the neural network converges at the same rate at all target frequencies as *spectral equilibrium*. In this state, the neural network can achieve a significant improvement in performance. In the following, we present the details of the Fourier Warm Start algorithm and demonstrate its effect with a numerical experiment.

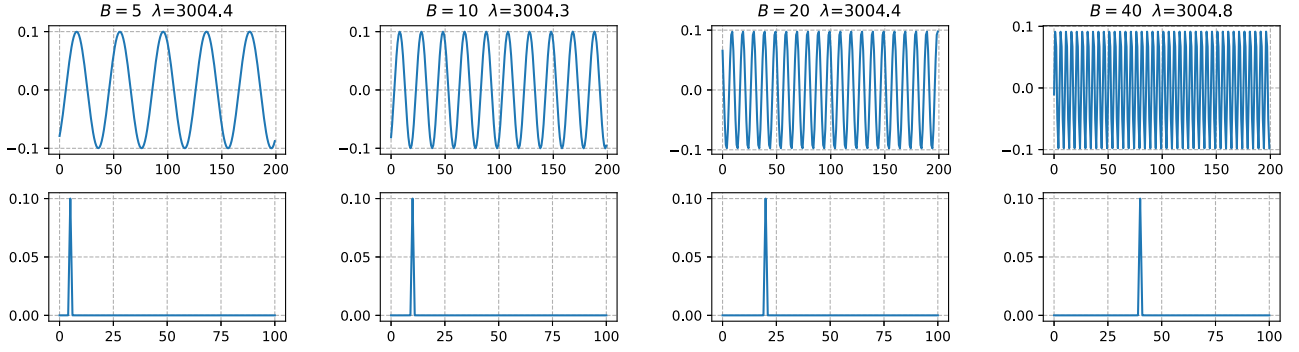


Fig. 3. Leading eigenfunctions for different initialization frequencies. (Top) The eigenfunctions correspond to the largest eigenvalues λ for initialization frequencies B . (Bottom) The spectrum information corresponds to the leading eigenfunctions. **Gist:** The principal frequency of the leading eigenfunction can be manipulated by adjusting the initialization value of the Fourier feature parameter.

NN Structure:

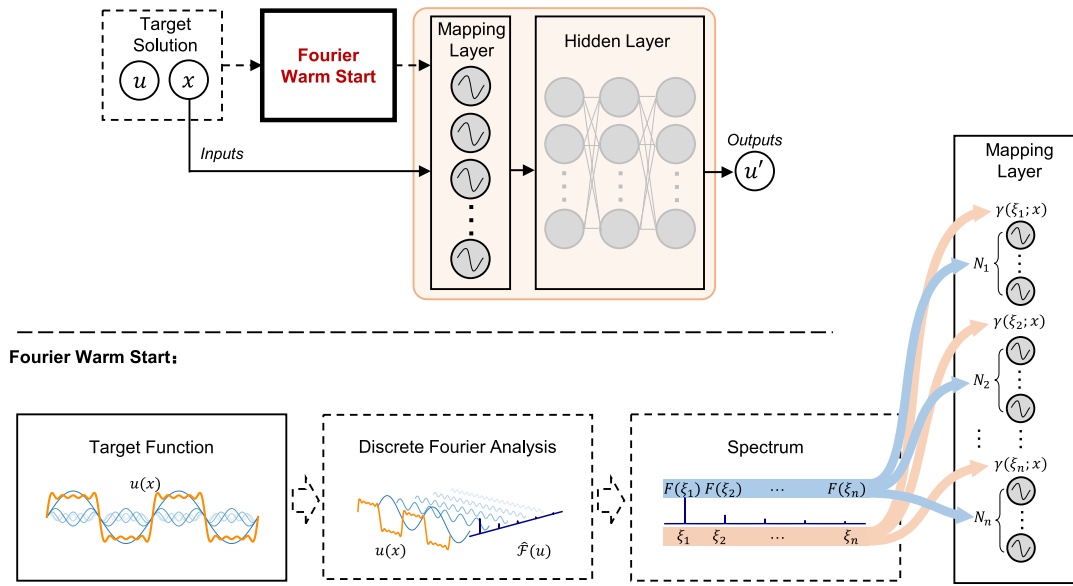


Fig. 4. Illustration of Fourier Warm Start method. The target solution $u(x)$ is first studied by a Discrete Fourier Transform (DFT) and the spectral information $F(\xi_c)$ is extracted. Then, the characteristic frequencies ξ_c and the corresponding magnitudes $|F(\xi_c)|$ are adopted to determine the values and neural numbers of the Fourier feature parameter $\mathbf{B}$, thus targeted initializing the Fourier mapping layer and performing the Fourier Warm Start method based on the specific target solution.

3.2.1. Fourier warm start algorithm

The Fourier Warm Start is a data-based targeted initialization method for Fourier feature neural networks (or its variants). The process is illustrated in Fig. 4. We consider a Fourier feature (FF) neural network. In conventional methods, the Fourier feature parameter $\mathbf{B}$ is randomly sampled from the Gaussian distribution $\mathcal{N}(0, \sigma^2)$. The standard deviation σ is a hyperparameter that is usually set empirically or requires a costly hyperparameter sweep or grid search. However, based on the approach introduced in Section 3.1, Fourier Warm Start can manipulate the initial values of the Fourier feature parameters according to the spectrum information of the target function. It basically has two steps, first studying the observations or known boundary/initial conditions of the target function with DFT and then embedding the extracted spectrum information into the Fourier feature mapping layer to initialize the network. The highlight of this method is that for different target solutions, the frequency of the dominant eigenfunction of the NTK is set to be the same as the characteristic frequency of the approximation of the target solution. At the same time, the corresponding dominant eigenvalues are adjusted according to the proportion of the spectrum information. As a result, the neural network can achieve a rapid and equilibrium convergence at all target frequencies. The details are presented as follows.

Let $D_s = \{v_1, \dots, v_N\} \subset D$ be an N -point discretization of the spatio-temporal domain D and $N \in \mathbb{N}$ is the total data size. Suppose a labeled data set $\{v_{j,k}, u_{j,k}\}_{k=1}^{n_j}$ is available for a finite collection of coordinate-solution pairs indexed by j and k , where n_j represents the number of the sampled points on dimension j . For forward problems, the labeled data set can be sampled from the initial conditions and at the boundary of the computational domain. For inverse problems, we consider two main types of tasks. The first is to predict the unknown coefficients in the governing equations based on the complete solution. The second is mainly focused on multiphysics problems, which require observations of one physical field to infer the values of other fields. Then, based on DFT, for labeled data $u_{j,k}$ on dimension $j = \{1, \dots, d\} \in \mathbb{Z}$, the spectral information $F_j \in \mathbb{C}^{n_j}$ can be extracted as

$$F_j(\xi_c) = \hat{F}(u_{j,k})_j = \sum_{k=1}^{n_j} u_{j,k} e^{-i \frac{2\pi}{n_j} c k}, \quad c = 1, \dots, n_j, \quad j = 1, \dots, d. \quad (21)$$

Let $a_{j,c} = |F_j(\xi_c)|$ be the magnitude, and $A_j = \sum_c |F_j(\xi_c)|$ be the sum of magnitudes. Then we leverage the spectrum information to manipulate the initialization value of the parameter $\mathbf{B}$ in the FF mapping function γ expressed in Eq. (6).

For $\mathbf{B}_j = \{B_{j,1}, \dots, B_{j,m}\}$ in dimension j , the values of the elements are sampled from a series of Gaussian distributions $\mathcal{N}(\xi_c, \epsilon^2)$ with the expectation ξ_c and the standard deviation ϵ as follows,

$$\begin{aligned} \underbrace{\{B_{j,1}, \dots, B_{j,m_1}\}}_{N_1} &\sim \mathcal{N}(\xi_1, \epsilon^2), \\ \underbrace{\{B_{j,m_1+1}, \dots, B_{j,m_2}\}}_{N_2} &\sim \mathcal{N}(\xi_2, \epsilon^2), \\ &\vdots \\ \underbrace{\{B_{j,m_{n_j}-1+1}, \dots, B_{j,m_n}\}}_{N_{n_j}} &\sim \mathcal{N}(\xi_{n_j}, \epsilon^2), \end{aligned} \quad (22)$$

Note that here ϵ represents a constant small value. This allows each set of elements to be specified to values close to the corresponding frequency but different from each other, thus ensuring the diversity of parameter initialization. The number of elements can be calculated by

$$N_c = \lfloor m \cdot a_{j,c} / A_j \rfloor \in \mathbb{N}, \quad (23)$$

and the operator $\lfloor \cdot \rfloor$ is the floor function representing the largest integer that is not greater than $(\cdot)$. The number of initialization elements is proportional to the magnitude of each initialization frequency, allowing the targeted initialization to capture the spectral relationship between the component frequencies of the labeled data. In practice, the sum of the calculated element number $\sum_c N_c$ may not be exactly equal to the total number m , so we specify N_ψ as the number of remaining uninitialized elements, and these elements can be initialized as

$$\underbrace{\{B_{j,m_n+1}, \dots, B_{j,m}\}}_{N_\psi} \sim \mathcal{N}(0, \epsilon^2). \quad (24)$$

At this point, all the elements for $\mathbf{B}_j$ in the dimension j have been initialized. Repeating the process for all d dimensions, we can finally initialize all parameters in $\gamma \in \mathbb{R}^{m \times d}$. The parameters of the remaining layers in Q_{FF} are initialized in a standard way, such as using the Xavier initialization (Glorot and Bengio, 2010). The entire process is also shown in Algorithm 1.

In particular, for all types of feature mapping described in Section 2.2, the above method can be applied directly. The parameters b in SF or CF mapping are still initialized with zero.

3.2.2. Validation study

We validate the proposed Fourier Warm Start method on numerical experiments. Let us consider a simple 1D multi-frequency ODE,

$$u(x)_{xx} + h^2 u(x) = f(x). \quad (25)$$

The analytical solution of this ODE is

$$u(x) = \sin(hx) + 0.5 \sin(5hx) + 0.2 \sin(20hx), \quad (26)$$

$f(x)$ can be specified by the analytical solution. We set $h = 2\pi$ and the computational domain is $x \in [0, 2]$. We first consider a data-driven problem and regress the target solution $u(x)$ with a neural network (NN) described in Eq. (15) with an architecture of 1-120-50-50-50-1, which means that it has four hidden layers, the first one with 120 units and the last three hidden layers with 50 units each. 200 labeled points are distributed evenly across the computational domain. We use the *tanh* activation and the network is trained with the Adam optimizer (Kingma and Ba, 2015) for 10k epochs. The learning rate is set to 3×10^{-4} . Fig. 1 shows the evolution of the normalized magnitudes at different frequencies and the predictions with their frequency distributions in the training process. It is apparent that for a fully-connected neural network, the network is more biased towards learning lower frequencies first, and higher frequencies require more training time or training epochs.

Algorithm 1 Fourier Warm Start Method.

Require: A labeled dataset including coordinate-solution pairs $\{\mathbf{v}, \mathbf{u}\} \in \mathbb{R}^d$. A neural network with a mapping layer $\gamma(\mathbf{v}; \mathbf{B})$, where $\mathbf{B} \in \mathbb{R}^{m \times d}$.

Ensure: A targeted initialized $\gamma(\mathbf{v}; \mathbf{B})$ with $\mathbf{B}$ manipulated based on the spectrum information of $\mathbf{u}$.

for $j = 1, \dots, d$ **do**

 Generate a sub-dataset $\{v_{j,k}, u_{j,k}\}_{k=1}^{n_j}$ on dimension j , where n_j represents the number of the sampled points on dimension j .

 Compute a discrete Fourier transform F_j of $u_{j,k}$: $F_j(\xi_c) = \hat{F}(u_{j,k})_j$

 Dividing $\mathbf{B}_j \in \mathbb{R}^m$ into n_j sets, where the c th set is $\{B_{j,m_{c-1}+1}, \dots, B_{j,m_c}\}$ with N_c elements.

for $c = 1, \dots, n_j$ **do**

 Compute the magnitude information: $a_{j,c} = |F_j(\xi_c)|$ and $A_j = \sum_c |F_j(\xi_c)|$

 Compute element number in c th set with a floor function: $N_c = \lfloor m \cdot a_{j,c} / A_j \rfloor \in \mathbb{N}$

 Specify the elements in c th set by sampling from Gaussian distribution $\mathcal{N}(\xi_c, \epsilon^2)$ with the expectation ξ_c and the standard deviation ϵ : $\{B_{j,m_{c-1}+1}, \dots, B_{j,m_c}\} \sim \mathcal{N}(\xi_c, \epsilon^2)$

end for

if $\sum_c N_c \neq m$ **then**

 Express the number of remaining elements as: $N_\psi = m - \sum_c N_c$

 Get the remaining elements: $\{B_{j,m_c+1}, \dots, B_{j,m}\} \sim \mathcal{N}(0, \epsilon^2)$

end if

end for

Next, we test the Fourier Warm Start method on the same numerical experiment. The neural network structure and all training settings are retained. Fig. 5 shows the training process after using the Fourier Warm Start method. It can be seen that the spectral bias is eliminated, and the neural network achieves a balanced convergence rate at the three target frequencies. We also consider a Fourier feature (FF) neural network with a default initialization setting described in Section 2.2 for comparison. From Fig. 6 we can find that with the Fourier Warm Start (FWS) method, the model achieves a relative L^2 error of 10^{-2} in only about 500 training epochs. This is much faster than the other two models. The final error for NN, FF, and FWS are 1.9×10^{-1} , 1.7×10^{-2} , and 4.6×10^{-4} , respectively. We also present the relative change of model parameters in Fig. 6, it clearly shows that with the FWS, the relative change of model parameters is the smallest. The results shown in Fig. 6 reflect the advantages of the proposed method, i.e. more reasonable initialization and better training performance.

The further performance comparison for the variants of FF mapping with and without Fourier Warm Start can be found in Appendix A. We find that the performance of the FF neural network and its variants such as SF and CF neural networks is quite similar. Therefore, in practice, the SF or CF mappings are more recommended due to their simpler structures. In addition, the performance of the FF neural network and its variants is significantly influenced by the initialization hyperparameter σ and only reaches the optimum when σ is tuned to a certain range. However, with the Fourier Warm Start method, the model can easily approach optimal performance for all feature mappings and does not rely on costly hyperparameter sweeps.

3.3. Fourier analysis boosted physics-informed neural network (Fab-PINN)

We present a novel network architecture that can effectively integrate the Fourier Warm Start for physics-informed problems, called Fourier Analysis Boosted Physics-Informed Neural Network (Fab-PINN).

The detailed architecture of Fab-PINN is presented in Fig. 7. A labeled dataset containing coordinate-solution pairs $\{\mathbf{v}, \mathbf{u}\} \in \mathbb{R}^d$ is first studied by the Fourier Warm Start method to initialize a Fourier

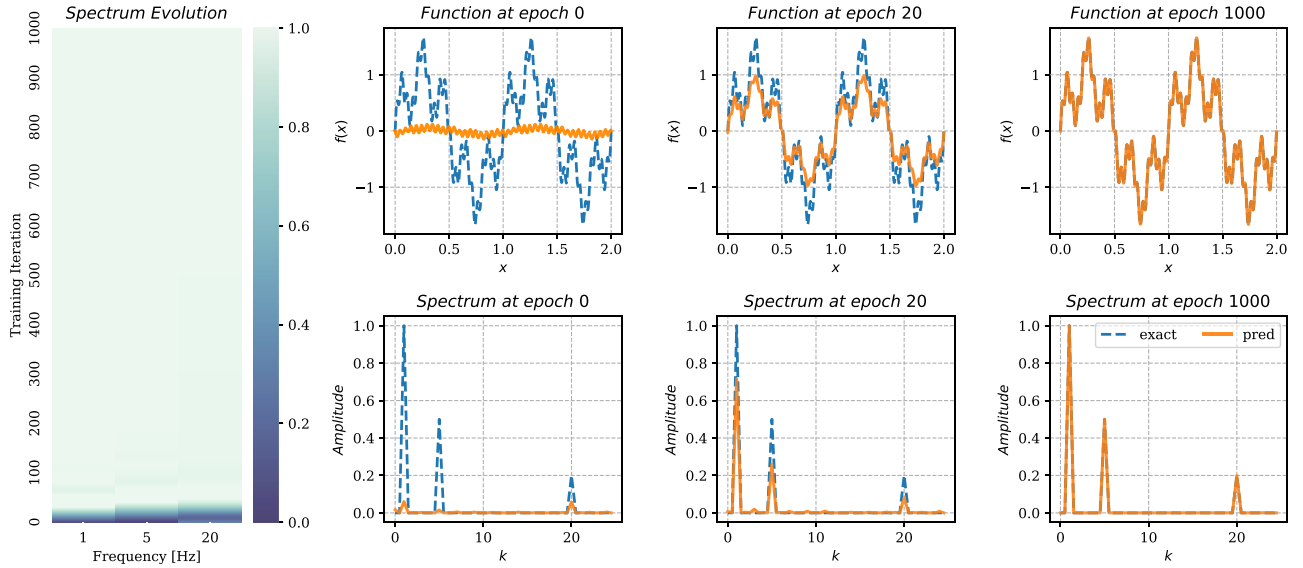


Fig. 5. Spectral equilibrium with Fourier Warm Start method. (Left) The evolution of the spectrum for different frequencies during training. The lighter color indicates that the spectrum at the corresponding frequency is closer to the target amplitude at the same frequency. (Right) The predictions and their frequency distributions of the network at epoch 0, 20, and 1k. **Gist:** The Fourier Warm Start method eliminates the *spectral bias* and the model reaches a *spectral equilibrium* in the training process. As a result, the performance of the model is greatly improved and the required training time (iterations) is considerably reduced.

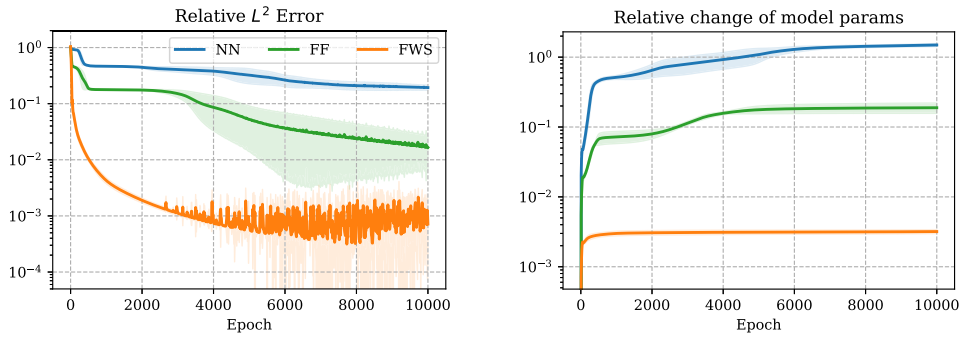


Fig. 6. Comparison of training history. (Left) Comparison of the relative L^2 error history of the fully-connected neural network (NN, in blue lines), the Fourier feature (FF, in green lines) neural network with default setting, and with the Fourier Warm Start (FWS, in orange lines) method. (Right) Comparison of the relative change of model parameters during training. All results are aggregated from 5 independent tests. **Gist:** With the Fourier Warm Start method, the model has achieved a significant performance improvement with the smallest variation of model parameters during training.

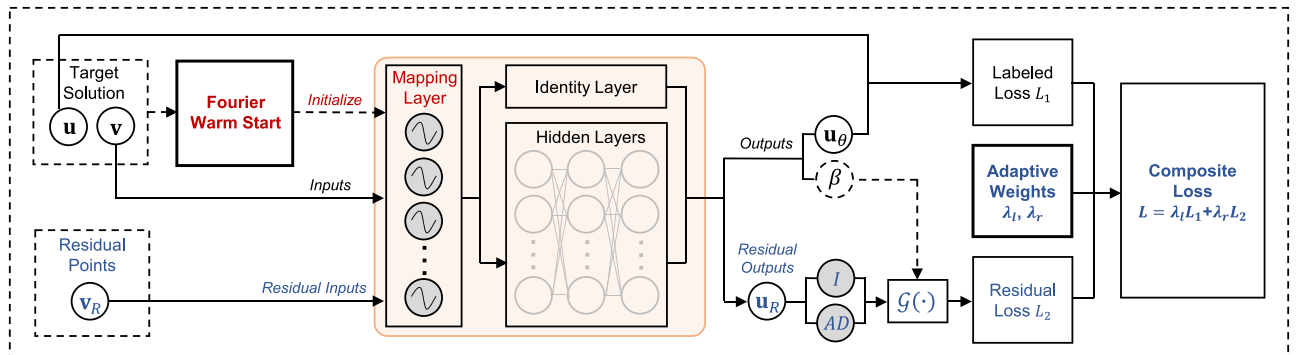


Fig. 7. Detailed architecture of *Fab-PINN*. *Fab-PINN* integrates the Fourier Warm Start algorithm and incorporates a composite loss. The labeled loss can be computed by the mean square error of the difference between the predicted and the exact value at initial points, boundary points, or other known data points in the computational domain. The residual loss is computed from the constraint equation at randomly or manually preferred selected residual points $\mathbf{v}_R$. The constraint equations $\mathcal{G}(\cdot)$ usually consist of the identity (I) of the predicted value $\mathbf{u}_R$ and its differentiation obtained by automatic differentiation (AD) methods. For some inverse problems, $\mathcal{G}(\cdot)$ may contain the unknown parameter β , which can also be approximated by the model in the training process. Furthermore, λ_l and λ_r are scaling factors to balance the composite loss in the training process.

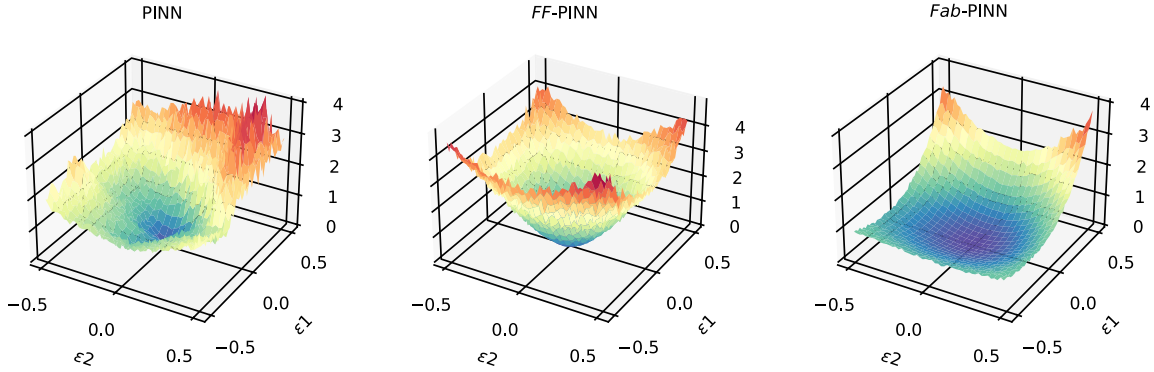


Fig. 8. Visualization of loss landscapes. The topology of loss landscapes for PINN, FF-PINN, and Fab-PINN separately. **Gist:** Fab-PINN leads to a smoother loss landscape compared with PINN and FF-PINN, thus accelerating the training process considerably.

feature mapping layer (or one of its variants in Section 2.2). We present the mapping layer with the Fourier Warm Start method as γ_{FWS} . The residual points $\mathbf{v}_R$ are randomly sampled or specified in the computational domain to constrain the governing equation $\mathcal{G}$. After the mapping layer, an identity layer I is placed in parallel with the hidden layers H_θ , directly connecting the output of the mapping layer and the hidden layers. θ represents the weights $\mathbf{W}$ and biases $\mathbf{b}$ of the hidden layers. Here, the identity layer is employed to enhance the effect of the Fourier Warm Start and can empirically further improve the final performance. The weight W_I and bias b_I of the identity layer are initialized as $1/N_m$ and zero respectively, where N_m is the number of Fourier feature parameters in the mapping layer. The detailed forward propagation can be defined as follows:

$$\begin{aligned} H_1 &= \phi \Sigma_1(\gamma_{FWS}(\mathbf{v}))_\theta = \phi(W_1 \cdot \gamma_{FWS}(\mathbf{v}) + b_1), \\ H_l &= \phi \Sigma_l(H_{l-1})_\theta = \phi(W_l \cdot H_{l-1} + b_l), \quad \text{for } l = 2, \dots, L-1, \\ \mathbf{u}_\theta &= I(\gamma_{FWS}(\mathbf{v})) + \Sigma_L(H_{L-1})_\theta \\ &= W_I \cdot \gamma_{FWS}(\mathbf{v}) + W_L \cdot H_{L-1} + b_I + b_L. \end{aligned} \quad (27)$$

With the predicted solution $\mathbf{u}_\theta$, the labeled loss can easily be obtained by a squared norm loss and the residual output $\mathbf{u}_R$ can also be calculated from the forward pass described in Eq. (27). To construct the residual loss, it is often necessary to express the constraint equation $\mathcal{G}$ in terms of the identity of the residual output $\mathbf{u}_R$ and its derivatives. The derivatives can be computed by the Automatic Differentiation (AD) method (Baydin et al., 2018). Finally, with the predicted solution $\mathbf{u}_\theta$ from Eq. (27) and the constructed constraint equation $\mathcal{G}(\mathbf{u}_R)$, we can obtain the composite loss function of Fab-PINN from Eq. (2).

In particular, $\mathcal{L}_1$ and $\mathcal{L}_2$ in the composite loss represent two types of constraints, labeled constraint from observation data and residual constraint from the underlying governing equations. Experience shows that there is an obvious competition between these two types of constraints in the optimization of the composite loss with the gradient descent methods (Wang et al., 2021a). The dominant constraint is more likely to bias the composite loss towards an inferior local minimum. Therefore, it is crucial to achieve a good trade-off between these two types of constraints for a successful physics-informed task. This trade-off can be usually achieved by using carefully selected or crafted values for the scaling factors λ_l and λ_r in the composite loss.²

To illustrate the advantage of the proposed method, we compare the results of baseline PINN, FF-PINN, and Fab-PINN.³ We visualize

² In this paper, we adopt the following Normalized Adaptive Weights (NAW) method to automatically calculate the weights λ_l and λ_r for all

Algorithm 2 Training procedure of Fab-PINN.

1. Build the structure of Fab-PINN based on Eq. (27).
 2. Initialize the parameters of the mapping layer based on FWS algorithm. Initialize other parameters with the Xavier scheme.
 3. Formulate the composite loss function $\mathcal{L}(\theta)$ with Eq. (2).
 4. Optimize the parameters θ of Fab-PINN with a gradient descent algorithm within S steps, at same time update the loss weights every K steps:
- for** $n = 1, \dots, S$ **do**
 Sample input points in the computational domain and calculate the loss $\mathcal{L}(\theta_n)$.
 if $n \bmod K = 0$ **then**
 Update loss weights with Eq. (28).
 end if
 Update θ via gradient descent algorithm
 $\theta_{n+1} = \theta_n - \eta \Delta_\theta \mathcal{L}(\theta_n)$.

end for

the loss landscapes in Fig. 8. The numerical experiment is formulated

physics-informed tasks,

$$\lambda_l = \frac{Tr_{min}}{Tr(\mathbf{K}_{uu}(t))}, \quad \lambda_r = \frac{Tr_{min}}{Tr(\mathbf{K}_{rr}(t))}, \quad (28)$$

where $Tr(\cdot)$ means the trace of the matrix and $Tr_{min} = \min(Tr(\mathbf{K}_{uu}(t)), Tr(\mathbf{K}_{rr}(t)))$. $\mathbf{K}_{uu}(t)$ and $\mathbf{K}_{rr}(t)$ are components in PINN's NTK matrix. Note that the proposed NAW method is modified from the work of Wang et al. (2022). It always keeps the maximum value of the weights at 1, thus allowing meaningful comparison of loss histories or loss landscapes between different tasks. Since computing the NTK matrix and its trace is computationally intensive, we can use a large update interval to avoid unnecessary computational overhead. In this paper, we update loss weights every 100 iterations for the first thousand iterations and then every 1k iterations by default, unless otherwise noted.

³ For baseline PINN, we use a fully connected neural network. For FF-PINN and Fab-PINN, the first hidden layer of the baseline PINN is replaced by a FF mapping layer with the same number of neurons. Xavier initialization (Glorot and Bengio, 2010) is used for all parameters in baseline PINN, and also for other parameters in FF-PINN and Fab-PINN, except for the Fourier feature parameters. The Fourier feature parameters in FF-PINN are initialized by the method introduced in Section 2.2, and in Fab-PINN they are initialized using the FWS.

Table 3

Training configurations used in the experimental study. For the model architecture, the neuron number of the first hidden layer is highlighted.

Experiment Governing Eqs.	Heat transfer Eq. (29)	Klein-Gordon Eq. (35)	Navier-Stokes Eq. (37)
Problem type	Forward	Inverse	Inverse
Model architecture	(x, t) -120-50-50-50- (u)	(x, t) -120-50-50-50- (u)	(x, y, t) -120-50-50-50-50- (u, v, p)
Total labeled points	1000	500×500	$256 \times 256 \times 50$ (for u, v) + 10 (for p)
Total residual points	20,000	20,000	20,000
Batch size	$N_{ic} = N_{bc} = N_r = 500$	$N_i = N_r = 500$	$N_{i1} = N_r = 500, N_{i2} = 10$
Initial learning rate	$1e-4$	$2e-3$	$2e-3$
Final learning rate	$1e-4$	$1e-5$	$1e-5$
Learning rate scheduler	Constant	Cosine Annealing	Cosine Annealing
Max. training iteration	10,000	100,000	200,000

Table 4

Computational cost of the numerical experiments. The training time and inference time shown are the *sum* of the time taken for 100 repetitions, and they do not include time for loss weight updates, learning rate updates, etc. The rise in *Fab*-PINN training and inference time as illustrated in the table is reasonable, given the substantial improvement in performance.

Computational cost	Heat transfer			Klein-Gordon			Navier-Stokes		
	Train (s)	Infer (s)	Total (h)	Train (s)	Infer (s)	Total (h)	Train (s)	Infer (s)	Total (h)
PINN	1.30	0.10	0.05	2.60	0.05	0.84	16.26	0.75	9.64
FF-PINN	1.49	0.19	0.06	2.73	0.06	0.87	17.17	1.18	10.31
<i>Fab</i> -PINN	1.89	0.33	0.08	3.30	0.07	1.07	18.15	1.34	10.89

in Eqs. (25) and (26), and all training settings are the same as in Section 3.2.2, except here we consider a physics-informed task. The loss landscapes are generated by perturbing the model parameters over the first and second Hessian eigenvectors at the end of the training (Yao et al., 2020). From Fig. 8, we can see that *Fab*-PINN leads to a much smoother loss landscape, significantly accelerating the training process.

4. Experiment results

In this section, we consider a series of challenging physics-informed tasks to demonstrate the effectiveness of the proposed method. First, we study a forward problem to solve the heat transfer equation and highlight the applicability of the proposed method in high-frequency conditions. Then we learn two inverse problems. The first inverse problem is to predict the unknown parameters in the multi-frequency Klein-Gordon equation. The second inverse problem is to solve the transient Navier-Stokes equation to derive the complete pressure field from the observed velocity field and very few pressure data. Finally, an ablation study is presented to illustrate the necessity of the proposed components in *Fab*-PINN. The experimental configurations are shown below in Table 3. Moreover, we summarize the computational cost for each task in Table 4. All the experiments are performed with Pytorch 1.13.0 on a single NVIDIA Quadro GV100 GPU with 32 GB memory.

4.1. Heat transfer equation

The heat transfer equation is a commonly used canonical benchmark problem to analyze the performance of conventional numerical methods and PINNs. We consider a one-dimensional transient heat transfer equation of the form

$$u_t(x, t) = \frac{k_1}{k_2} u_{xx}(x, t) \quad \text{in } \Omega \times [0, T], \quad (29)$$

where the spatial domain is $\Omega = [0, 1]$ and $T = 1$. The theoretical solution to this benchmark problem is

$$u(x, t) = e^{-k_1 t} \sin(k_2 x). \quad (30)$$

In this task, the forward problem is considered, which means that the known labeled points are only sampled from the initial and boundary conditions of the computational domain. The initial and boundary

conditions can be easily derived from the theoretical solution and are given by

$$u(x, 0) = \sin(k_2 x), \quad x \in [0, 1] \quad (31)$$

$$u(0, t) = u(1, t) = 0, \quad t \in [0, 1]. \quad (32)$$

We take $k_1 = 2.0$ and $k_2 = 100\pi$. The model architecture and training settings are shown in Table 3. The composite loss function of this task can be approximated by discrete collocation points randomly sampled in appropriate computational domains, which is given by

$$\begin{aligned} \mathcal{L}(\theta) &= \lambda_{l1} \mathcal{L}_{ic}(\theta) + \lambda_{l2} \mathcal{L}_{bc}(\theta) + \lambda_r \mathcal{L}_r(\theta) \\ &= \frac{\lambda_{l1}}{N_{ic}} \sum_{i=1}^{N_{ic}} |u_\theta(x_i, 0) - u_{ic}(x_i, 0)|^2 + \frac{\lambda_{l2}}{N_{bc}} \sum_{i=1}^{N_{bc}} |u_\theta(x_i, t_i) - u_{bc}(x_i, t_i)|^2 \\ &\quad + \frac{\lambda_r}{N_r} \sum_{i=1}^{N_r} |r_\theta(\hat{x}_i, \hat{y}_i)|^2 \\ &= \frac{\lambda_{l1}}{N_{ic}} \sum_{i=1}^{N_{ic}} |u_\theta(x_i, 0) - u_{ic}(x_i, 0)|^2 + \frac{\lambda_{l2}}{N_{bc}} \sum_{i=1}^{N_{bc}} |u_\theta(x_i, t_i) - u_{bc}(x_i, t_i)|^2 \\ &\quad + \frac{\lambda_r}{N_r} \sum_{i=1}^{N_r} \left| \frac{\partial u_\theta(\hat{x}_i, \hat{t}_i)}{\partial \hat{t}_i} - \frac{k_1}{k_2} \frac{\partial^2 u_\theta(\hat{x}_i, \hat{t}_i)}{\partial \hat{x}_i^2} \right|^2 \end{aligned} \quad (33)$$

where x, y are labeled coordinate points and $\hat{x}, \hat{y}$ are residual coordinate points. We choose batch sizes $N_{ic} = N_{bc} = N_r = 500$ for each iteration of the gradient descent. And the composite loss function is optimized by the Adam optimizer for 10k iterations with a learning rate of 1×10^{-4} . The loss weights are also updated in the training process according to Eq. (28). For *Fab*-PINN, the labeled data distributed over the initial and boundary conditions are studied to employ the FWS method.

We compare the performance of baseline PINN, *FF*-PINN, and *Fab*-PINN and present the initialization results, final predictions, and point-wise error in Figs. 9(a), 9(b) and 9(c), respectively. Focusing on the initialization results, we can intuitively perceive that the pattern of initial results of baseline PINN and *FF*-PINN seems too *sparse* compared to the exact solution. However, for *Fab*-PINN, there is a certain similarity between the pattern of initial results and the exact results. This phenomenon actually reflects the essence that only the initialization of *Fab*-PINN successfully captures the characteristic frequency information

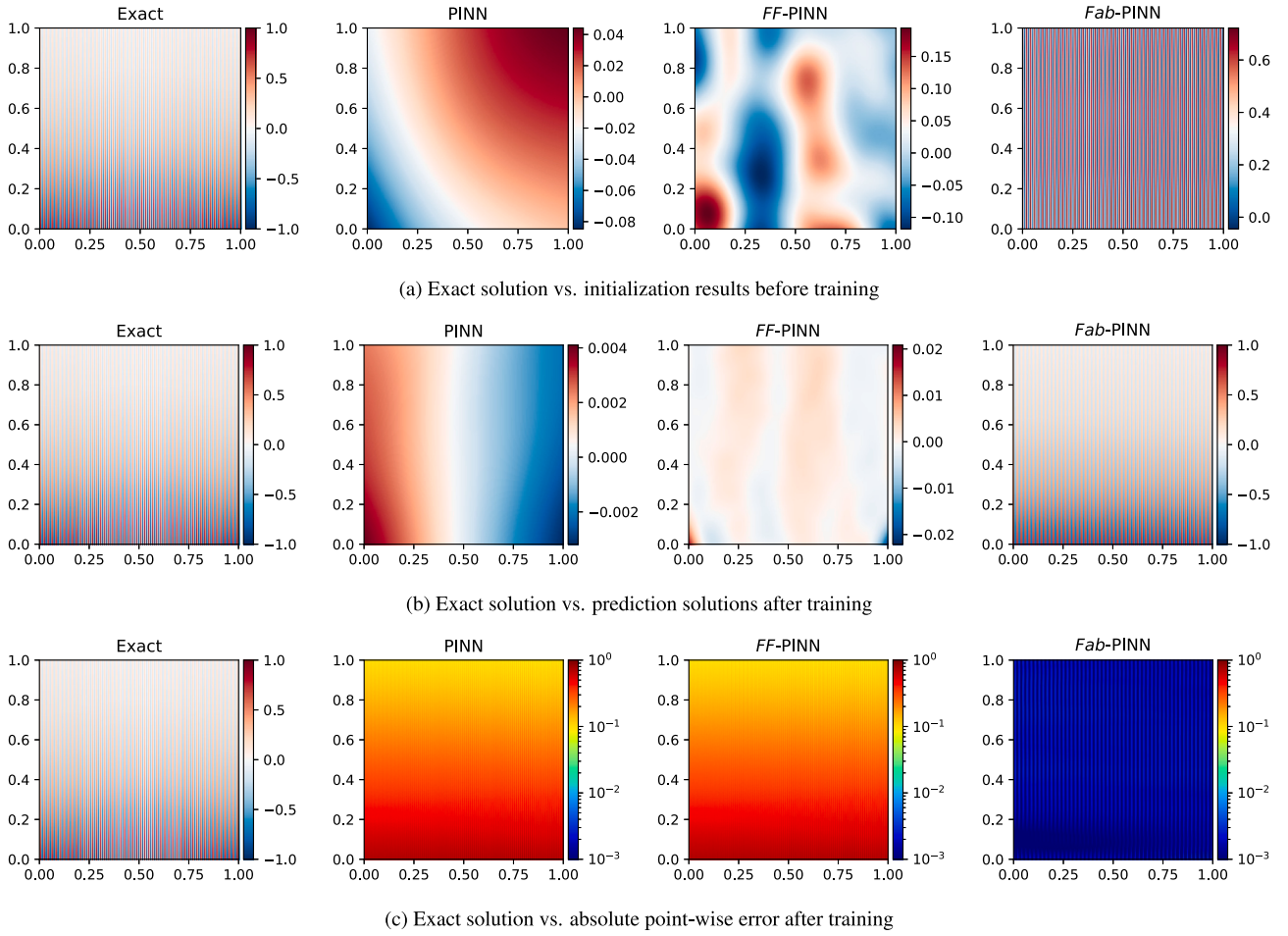


Fig. 9. Heat transfer equation. (Top) Exact solution compared to the initialization result of baseline PINN, *FF*-PINN, and *Fab*-PINN. (Middle) Exact solution compared to the predicted solution of baseline PINN, *FF*-PINN, and *Fab*-PINN. (Bottom) Exact solution compared to absolute point-wise error for the predicted solutions of baseline PINN, *FF*-PINN, and *Fab*-PINN. The relative L^2 errors are 9.99×10^{-1} , 9.99×10^{-1} , and 4.40×10^{-3} , respectively. The results are aggregated from 5 independent cases.

Table 5

High-frequency heat equation. Comparison of relative L^2 error averaged from 5 independent cases.

Method	PINN	<i>FF</i> -PINN	ST-MFF (def)	ST-MFF (tuned)	<i>Fab</i> -PINN
Error	1.00e+00	9.99e-01	1.00e+00	2.06e-03	1.59e-03

of the target solution, thus significantly improving the model performance. This is confirmed by the final results and the point-wise error. It can be seen that both baseline PINN and *FF*-PINN fail in this task and the predictions deviate significantly from the exact solution. For *Fab*-PINN, on the other hand, the predictions are in much better agreement with the exact solution. The final relative L^2 error, aggregated from five independent cases, is 9.99×10^{-1} , 9.99×10^{-1} , and 4.40×10^{-3} for baseline PINN, *FF*-PINN, and *Fab*-PINN respectively.

To further demonstrate the performance of the proposed method on high-frequency problems, we compare it to a PINN architecture termed ST-MFF for high-frequency tasks presented in Wang et al. (2021b). We set $k_1 = 1$, $k_2 = 500\pi$, and the total number of training iterations to $40k$. The initial learning rate is 1×10^{-3} , we employ an exponential learning rate decay with a decay rate of 0.9 every thousand iterations. The network structure is set as (x, t) -100-100-100- (u) and we use batch sizes $N_{ic} = N_{bc} = N_r = 128$ for each iteration of the gradient descent. All training settings are consistent with those in (Wang et al., 2021b). For ST-MFF, the spatial and temporal input coordinates are embedded by two separate Fourier features with different initialization

hyperparameters σ_x and σ_t . We test default ST-MFF with $\sigma_x = \sigma_t = 1$ and tuned ST-MFF with $\sigma_x = 200$, $\sigma_t = 1$ in this task. The final relative L^2 error for baseline PINN, *FF*-PINN, default ST-MFF, tuned ST-MFF, and *Fab*-PINN averaged from 5 independent cases is shown in Table 5. We can see that baseline PINN, *FF*-PINN, and default ST-MFF all failed in this task, while *Fab*-PINN can effectively handle high-frequency problems and has the best performance among all methods. Noted that although the tuned ST-MFF can provide good accuracy in this task, finding the optimally tuned σ_x and σ_t usually requires a costly hyperparameter sweep. However, with the FWS method, the FF parameters are automatically initialized to reasonable values, avoiding the costly tuning of any hyperparameters.

4.2. Klein-Gordon equation

In this example, we would like to highlight the ability of the proposed method to tackle inverse problems. We study the transient Klein-Gordon equation, an important PDE with widespread applications in quantum physics, solid-state physics, and many other scientific fields. One spatial dimension transient Klein-Gordon equation is given by

$$u_{tt}(x, t) - u_{xx}(x, t) + \alpha u(x, t) + u^k = q(x, t; \beta), \quad \text{in } \Omega \times [0, T], \quad (34)$$

where the spatial domain is $\Omega = [-1, 1]^2$, $T = 1$ and $q(x, y)$ is the source term. We consider the following coefficients in the equation as known constants, including $\alpha = 0$ and $k = 3$. A fabricated analytical solution is adopted to assess the performance, which is given by

$$u = x \sin(5\pi x) \cos(10\pi t) + t \sin(40\pi x) \cos(40\pi t) + \beta x t, \quad (35)$$

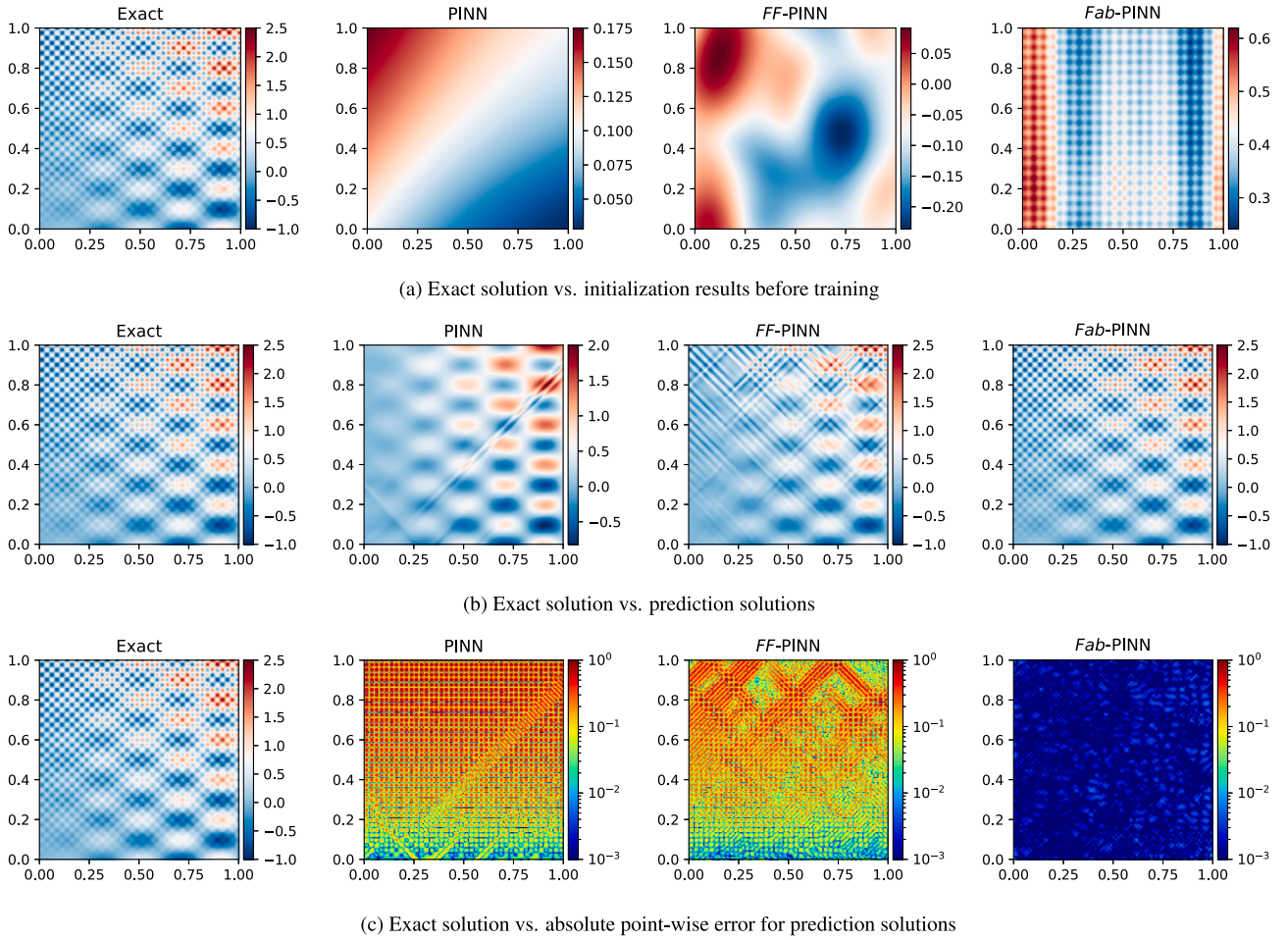


Fig. 10. Klein-Gordon equation. (Top) Exact solution versus the initialization result of baseline PINN, *FF*-PINN, and *Fab*-PINN. (Middle) Exact solution versus the predicted solution of baseline PINN, *FF*-PINN, and *Fab*-PINN. (Bottom) Exact solution versus absolute point-wise error for the predicted solutions of baseline PINN, *FF*-PINN, and *Fab*-PINN. The relative L^2 errors are 5.43×10^{-1} , 3.29×10^{-1} and 2.55×10^{-3} , respectively. The results are aggregated from 5 independent cases.

and the source term $q(x, t; \beta)$ can be determined from the analytical solution. Here β in the governing equation is an unknown parameter. We would like to learn the value of β from the analytical solution and compare the prediction with the exact value $\beta = 1$.

The composite loss function computed from discrete collocation points can be given by

$$\begin{aligned}
 \mathcal{L}(\theta) &= \lambda_l \mathcal{L}_l(\theta) + \lambda_r \mathcal{L}_r(\theta) \\
 &= \frac{\lambda_l}{N_l} \sum_{i=1}^{N_l} |u_\theta(x_i, t_i) - u_i|^2 + \frac{\lambda_r}{N_r} \sum_{i=1}^{N_r} |r_\theta(\hat{x}_i, \hat{t}_i; \hat{\beta})|^2 \\
 &= \frac{\lambda_l}{N_l} \sum_{i=1}^{N_l} |u_\theta(x_i, t_i) - u_i|^2 \\
 &\quad + \frac{\lambda_r}{N_r} \sum_{i=1}^{N_r} \left| \frac{\partial^2 u_\theta(\hat{x}_i, \hat{t}_i)}{\partial \hat{x}_i^2} - \frac{\partial^2 u_\theta(\hat{x}_i, \hat{t}_i)}{\partial \hat{x}_i^2} + u_\theta^3(\hat{x}_i, \hat{t}_i) - q_\theta(\hat{x}_i, \hat{t}_i; \hat{\beta}) \right|^2
 \end{aligned} \tag{36}$$

where x, t are labeled spatio-temporal coordinate points, $\hat{x}, \hat{t}$ are residual spatio-temporal coordinate points and $\hat{\beta}$ is the prediction of the unknown parameters. We continue to choose batch sizes $N_l = N_r = 500$ for each iteration of gradient descent and optimize the model using the Adam optimizer for 100k iterations. And the learning rate is updated with an initial value of 2×10^{-3} and a final value of 1×10^{-5} according to the Cosine Annealing schedule. The model architecture and training settings are summarized in Table 3. For *Fab*-PINN, in order to use FWS, we need to select a set of labeled data located on the same line in each dimension to extract the frequency information of different dimensions. Here, the positions of the two lines are $x = 0.5$ and $y = 0.5$. The selection

Table 6

Klein-Gordon equation. Comparison of relative L^2 error for β aggregated from 5 independent cases.

Method	Exact	Prediction	Error
PINN	1.00e+00	1.44e-02	9.86e-01
<i>FF</i> -PINN	1.00e+00	-4.99e-01	1.50e+00
<i>Fab</i> -PINN	1.00e+00	9.46e-01	5.35e-02

of the positions is not unique, and the interpolation method can be employed if the observations are randomly distributed.

For this problem, the governing equations are more complicated, and the weight of the high-frequency components in the exact solution gradually increases with time. We test 5 independent cases, and the mean relative L^2 error of u is shown in Fig. 10. As for the prediction of the unknown parameter β , the comparison of the relative L^2 error for baseline PINN, *FF*-PINN, and *Fab*-PINN can be found in Table 6. We can see that for this multi-frequency transient Klein-Gordon equation, both baseline PINN and *FF*-PINN fail to predict the unknown parameter, while *Fab*-PINN gives a significantly more accurate prediction.

4.3. Transient Navier-Stokes equations

In this section, we study a multi-physics inverse problem described by the Navier-Stokes equations. The Navier-Stokes equations are a set of nonlinear, coupled partial differential equations that formulate

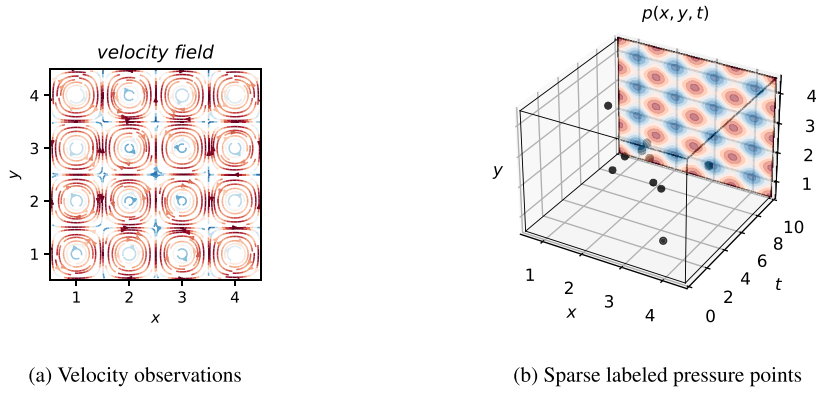


Fig. 11. Known labeled data. The total labeled data consists of dense velocity observations and very sparse pressure information (10 randomly sampled pressure points in this task). Highly accurate pressure information for the entire flow field is derived without any initial or boundary conditions in this task considering engineering scenarios.

a fundamental model for understanding the behavior of fluids and play a crucial role in numerous scientific and engineering disciplines, including meteorology, aerodynamics, and chemical engineering.

Two-dimensional transient incompressible Navier–Stokes equations with the continuity equation are given explicitly by

$$u_x + v_y = 0, \quad (37a)$$

$$u_t + (uu_x + vu_y) = -p_x + \frac{1}{Re}(u_{xx} + u_{yy}), \quad (37b)$$

$$v_t + (uv_x + vv_y) = -p_y + \frac{1}{Re}(v_{xx} + v_{yy}), \quad (37c)$$

where $u(x, y, t)$, $v(x, y, t)$ and $p(x, y, t)$ denote the x and y -component of the velocity, and the pressure field, respectively. Re represents the Reynolds number, which is a dimensionless parameter that describes the ratio of inertial forces to viscous forces in a fluid flow. To evaluate the accuracy of our models, we consider a well-established benchmark problem, the Taylor–Green vortex with $Re = 100$. This problem describes the evolution of decaying vortices in a transient flow. The analytical solution can be expressed in closed form as follows:

$$u(x, y, t) = -\cos(\pi x) \sin(\pi y) \exp(-2\pi^2 t / Re), \quad (38a)$$

$$v(x, y, t) = \sin(\pi x) \cos(\pi y) \exp(-2\pi^2 t / Re), \quad (38b)$$

$$p(x, y, t) = -\frac{\rho}{4} [\cos(2\pi x) + \cos(2\pi y)] \exp(-4\pi^2 t / Re). \quad (38c)$$

The spatial–temporal domain of the problem is considered as $x \in [0.5, 4.5]$, $y \in [0.5, 4.5]$, $t \in [0, 10]$.

In practical engineering or physical experiment scenarios, it is relatively straightforward to obtain high-precision velocity information for a flow field, such as through the Particle Image Velocimetry (PIV) method (Scarano, 2012), but it is quite challenging to obtain a pressure field. In general, we only have access to a small number of pressure measurements at critical points. Given the background, we would like to validate the ability of the proposed method to derive full-field solutions from flow field observations. This effectively requires solving an ill-posed problem where dense velocity information is already known, with very sparse pressure information (i.e. 10 randomly sampled pressure points in this task) to infer the pressure for the entire flow field without any initial or boundary conditions. The composite loss function computed from discrete collocation points can be given by

$$\mathcal{L}(\theta) = \lambda_{l1} \mathcal{L}_{l1}(\theta) + \lambda_{l2} \mathcal{L}_{l2}(\theta) + \lambda_r \mathcal{L}_r(\theta), \quad (39)$$

and the labeled loss and the residual loss can be expressed specifically as

$$\mathcal{L}_{l1}(\theta) = \frac{1}{N_{l1}} \left(\sum_{i=1}^{N_{l1}} |u_\theta(x_i, y_i, t_i) - u_i|^2 + \sum_{i=1}^{N_{l1}} |v_\theta(x_i, y_i, t_i) - v_i|^2 \right), \quad (40a)$$

$$\mathcal{L}_{l2}(\theta) = \frac{1}{N_{l2}} \sum_{i=1}^{N_{l2}} |p_\theta(x_i, y_i, t_i) - p_i|^2, \quad (40b)$$

$$\mathcal{L}_r(\theta) = \frac{1}{N_r} \left(\sum_{i=1}^{N_r} |r_{\theta1}(\hat{x}_i, \hat{y}_i, \hat{t}_i)|^2 + \sum_{i=1}^{N_r} |r_{\theta2}(\hat{x}_i, \hat{y}_i, \hat{t}_i)|^2 + \sum_{i=1}^{N_r} |r_{\theta3}(\hat{x}_i, \hat{y}_i, \hat{t}_i)|^2 \right), \quad (40c)$$

where x, y, t are labeled spatio-temporal coordinate points and $\hat{x}, \hat{y}, \hat{t}$ are residual spatio-temporal coordinate points. The PDE constraints are specified by

$$\begin{aligned} r_{\theta1} &= \frac{\partial u_\theta}{\partial t} + u_\theta \frac{\partial u_\theta}{\partial x} + v_\theta \frac{\partial u_\theta}{\partial y} + \frac{\partial p_\theta}{\partial x} - \frac{1}{Re} \left(\frac{\partial^2 u_\theta}{\partial x^2} + \frac{\partial^2 u_\theta}{\partial y^2} \right), \\ r_{\theta2} &= \frac{\partial v_\theta}{\partial t} + u_\theta \frac{\partial v_\theta}{\partial x} + v_\theta \frac{\partial v_\theta}{\partial y} + \frac{\partial p_\theta}{\partial y} - \frac{1}{Re} \left(\frac{\partial^2 v_\theta}{\partial x^2} + \frac{\partial^2 v_\theta}{\partial y^2} \right), \\ r_{\theta3} &= \frac{\partial u_\theta}{\partial x} + \frac{\partial v_\theta}{\partial y}. \end{aligned} \quad (41)$$

We choose the batch sizes $N_{l1} = 500$, $N_{l2} = 10$, $N_r = 500$. Note that for velocity data and residual data, the data points are randomly sampled at each iteration of the gradient descent. But for pressure data, 10 points in the computational domain are randomly selected as known data points before the start of the training, and their positions are fixed in the same case, as shown in Fig. 11(b).

In the training process, the learning rate is updated according to a Cosine Annealing schedule with an initial value of 2×10^{-3} and a final value of 1×10^{-5} . The full field information is predicted by the model after 200k iterations of training. We summarize the model architecture and training settings in Table 3. Note that for *Fab*-PINN, the velocity observations over three lines are studied by FWS to extract the frequency information of different dimensions, which are $y = 1, t = 0$ for dimension x , $x = 1, t = 0$ for dimension y , and $x = 1, y = 1$ for dimension t . The selection of the line positions is not unique, and the interpolation method can be employed if the observations are randomly distributed.

The relative L^2 errors of the respective pressure predictions are 6.50×10^{-1} , 8.13×10^{-2} and 9.55×10^{-4} for baseline PINN, *FF*-PINN, and *Fab*-PINN, respectively. Fig. 12 shows the pressure predictions and the point-wise error at $t = 10$. The results highlight that *Fab*-PINN can handle this ill-posed problem of extremely sparse pressure input while other conventional models cannot.

4.4. Ablation study of *Fab*-PINN

To illustrate the necessity of the components in *Fab*-PINN, we perform a series of comparative tests listed in Table 7. The tested components include Hidden Layers (HL), Normalized Adaptive Weights (NAW) method, Mapping Layer (ML), Fourier Warm Start (FWS), and Identity Layer (IL). M1 adopts the PINN model without dynamically

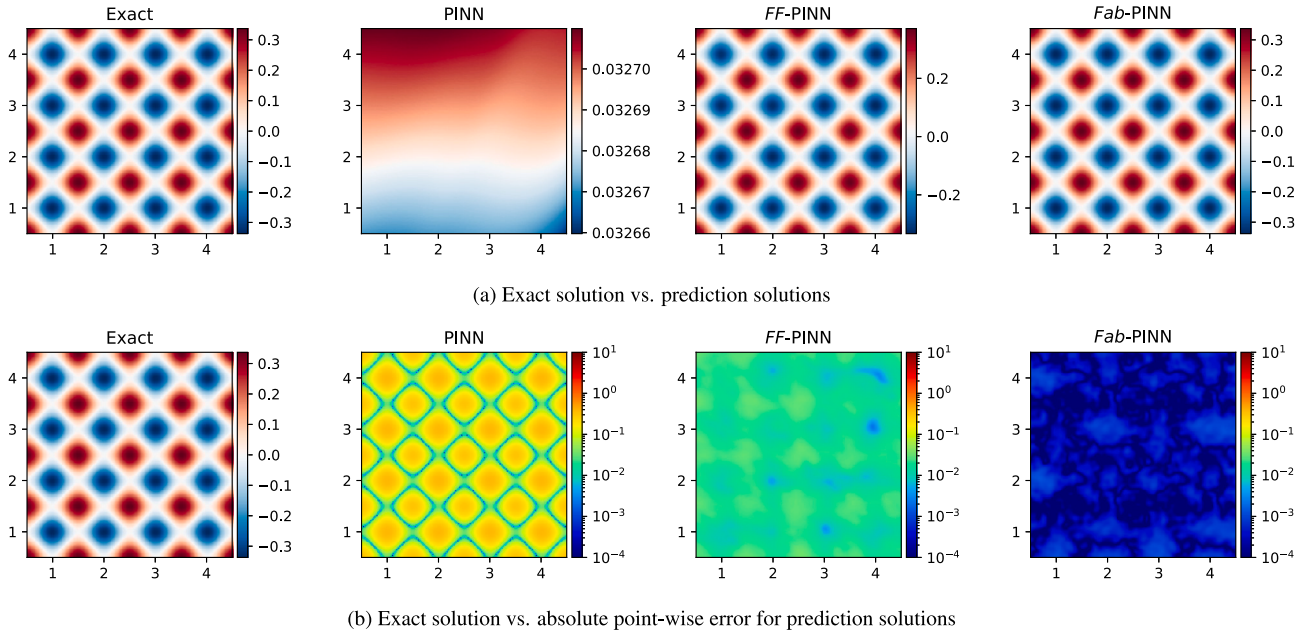


Fig. 12. Navier–Stokes equations. (Top) Exact solution versus the predicted solution of baseline PINN, *FF*-PINN, and *Fab*-PINN. (Bottom) Exact solution versus absolute point-wise error for the predicted solutions of baseline PINN, *FF*-PINN, and *Fab*-PINN. The relative L^2 errors are 6.50×10^{-1} , 8.13×10^{-2} and 9.55×10^{-4} , respectively. The results are aggregated from 5 independent cases.

Table 7

Ablation study results. The relative L^2 Error of u and the unknown parameter β . All results shown are from the average of 5 independent trials. In particular, the model in M2, M4, and M8 are the same as the baseline PINN, *FF*-PINN, and *Fab*-PINN mentioned in the previous sections.

Model	HL	NAW	ML	FWS	IL	Forward task		Inverse task	
						E_u	E_u	E_β	E_β
M1	✓	–	–	–	–	1.00e+00	2.69e+00	9.71e–01	9.71e–01
M2	✓	✓	–	–	–	9.99e–01	5.43e–01	9.86e–01	9.86e–01
M3	✓	–	✓	–	–	1.00e+00	5.72e+00	1.44e+00	1.44e+00
M4	✓	✓	✓	–	–	9.99e–01	3.29e–01	1.50e+00	1.50e+00
M5	✓	–	✓	✓	–	4.87e–03	2.69e–03	9.40e–02	9.40e–02
M6	✓	✓	✓	✓	–	4.55e–03	2.61e–03	1.10e–01	1.10e–01
M7	✓	–	✓	✓	✓	4.52e–03	2.64e–03	6.50e–02	6.50e–02
M8	✓	✓	✓	✓	✓	4.40e–03	2.55e–03	5.35e–02	5.35e–02

adapting the loss weights. The loss weights are calculated at the first epoch according to Eq. (28) and are kept fixed throughout the training process. All of the following models that do not use the NAW method set the loss weights in the same way. In M2, the proposed NAW method is used to automatically calculate the loss weights in the training process. The model adopted in M2 is the same as the baseline PINN model used for comparison in Sections 4.1, 4.2, and 4.3. In M3 and M4, a Fourier feature mapping layer replaces the first hidden layer of the PINN model in M1 and M2, respectively. The model in M4 is the same as the *FF*-PINN model mentioned in the previous sections. Next, the FWS method is applied. In particular, M5 and M6 demonstrate the specific effects of the FWS method. The complete *Fab*-PINN model is used in M8.

In the ablation study, both forward and inverse tasks are considered. In the forward task, the heat transfer equation formulated as Eq. (29) is studied. The parameters in the governing equation are assumed to be $k_1 = 2.0$ and $k_2 = 100\pi$. In the inverse task, the Klein–Gordon equation described in Eq. (34) is studied. The model architecture and detailed experimental settings are retained as shown in Table 3. Five independent trials are run for each test and the final results are averaged. The experimental results shown in Table 7 indicate that *Fab*-PINN with the complete structure (M8) achieves the best accuracy in both forward and inverse tasks. In addition, we can see that starting from M5, the model performance has noticeable improvement. This indicates the significant effect of the proposed FWS method.

5. Discussion

In this work, we first present an approach to manipulate the NTK eigenvalues of the Fourier feature neural network. Then, we propose the FWS method to automatically balance the convergence rate of the neural network at different frequencies, thereby mitigating the spectral bias and improving the performance. Moreover, for physics-informed tasks, we propose a novel integrated architecture called *Fab*-PINN, which provides a unified training strategy to address the challenges in training PINNs, including the spectral bias limitation and the non-equilibrium convergence in the composite loss. We further validate the effectiveness of the proposed method through a series of challenging experiments. The results demonstrate an improvement in accuracy of several orders of magnitude over previous methods.

Despite the exciting progress, there are still some limitations to the proposed method. The main limitation is that for the FWS algorithm, the labeled data must contain valid frequency information for the target function. This condition is usually satisfied in a wide range of inverse problems. However, in certain forward problems, if the known initial or boundary conditions do not contain sufficient frequency information, the proposed method will not achieve the desired performance. In this case, the conventional initialization method for Fourier feature neural networks based on empirical or hyperparameter sweep should still be used. We hope our work can provide a new perspective to further improve the performance of PINNs.

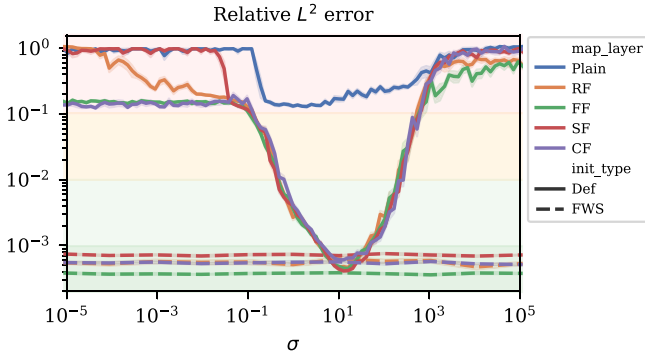


Fig. A.13. Comparison between FF mapping and its variants with/without Fourier Warm Start (FWS) method. Different colors indicate different mapping layers. *Plain* here means fully-connected neural networks with no mapping layer. RF, FF, SF, and CF represent random Fourier feature mapping, trainable Fourier feature mapping, sinusoidal feature mapping, and cosinusoidal feature mapping, respectively. The solid lines represent the adoption of the default initialization strategy, where the Fourier feature parameters are randomly sampled from a Gaussian distribution $\mathcal{N}(0, \sigma^2)$ with a hyperparameter σ . The dashed lines indicate training with the FWS method. Results were obtained by training a fully-connected neural network with the structure 1-120-50-50-50-1 over 10k iterations of gradient descent. The Adam optimizer and Cosine Annealing learning rate schedule with an initial value of 2×10^{-3} and a final value of 1×10^{-5} are employed. The final results are aggregated from 5 independent cases.

CRedit authorship contribution statement

Ge Jin: Conceptualization, Methodology, Software, Writing – original draft. **Jian Cheng Wong:** Software, Writing – review & editing, Resources. **Abhishek Gupta:** Validation, Investigation, Writing – review & editing. **Shipeng Li:** Supervision, Writing – review & editing. **Yew-Soon Ong:** Supervision, Project administration, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

The authors acknowledge the financial support received from the China Scholarship Council (202206030081). The work was supported in part by the Indian Science & Engineering Research Board Ramanujan Fellowship grant RJF/2022/000115.

Appendix A. Comparison of FF mapping variants with and without Fourier Warm Start

We further compare the performance of various feature mappings described in Section 2.2, through the numerical multi-frequency ODE experiment in Section 3.2.2.

The effect of the hyperparameter σ is studied in a wide range of values for different feature mapping layers with and without the Fourier Warm Start method. One hundred and one σ values uniformly distributed on a logarithmic scale in the interval from 10^{-5} to 10^5 are investigated, and the results are shown in Fig. A.13. The solid lines represent the networks initialized by the conventional initialization method described in Section 2.2 without Fourier Warm Start, while the

dashed lines are the opposite. It is apparent that with the conventional initialization strategy, the results for FF neural networks and their variants vary dramatically as σ changes, with the final errors differing by several orders of magnitude.

Moreover, all feature mapping layers can significantly improve the performance of the networks compared to the full-connected neural network when σ values are within a certain range. And interestingly, in this range, the behavior of FF mapping and its variants is quite similar. Although all the feature mapping layers can provide a huge performance improvement if σ is tuned to an appropriate range, this hyperparameter range becomes quite narrow if we want to obtain high-accuracy results in this task. Note that this narrow range can be variable for different specific tasks or datasets, and usually requires a costly hyperparameter sweep to finally determine it (Wong et al., 2022). However, with Fourier Warm Start, all feature mapping layers are automatically tailored to a specific target function and thus can easily provide a high accuracy performance that is close to or even better than the optimally tuned results without Fourier Warm Start. This greatly improves the accessibility of high-performance mapping layers and increases the potential of Fourier feature neural networks and their variants.

Appendix B. Experiment results for image regression

In this section, we provide experimental results from image regression task to demonstrate the effectiveness of the proposed method on targets with more continuous frequency distributions.

We obtain the results by training a neural network with the structure 1-512-50-50-50-50-1 over 10k iterations of gradient descent. Note that here we use a wide mapping layer to better capture the continuously distributed eigenfrequencies of the target. The Adam optimizer and a constant learning rate of value 5×10^{-4} are employed. From Fig. B.14, it is apparent that the proposed FWS method captures image details more accurately. This can be attributed to the fact that FWS effectively mitigates the pathology of spectral bias, enabling the neural network to learn information on different frequencies in a more balanced way.

We also show the MSE loss history and Peak Signal-to-Noise Ratio (PSNR) history in Fig. B.15. We can see that the FWS method achieves the smallest MSE and best PSNR among all the methods, substantiating the applicability of the proposed method in scenarios with continuous frequency distribution.

Appendix C. The eigenvalues and eigenfunctions of a two-layer bias-free neural network

Fourier feature (FF) neural network. The proof is based on the work in Wang et al. (2021b), here we adopt a special initialization strategy and generalized the conclusions.

Proof. Consider the simplified Fourier feature neural network described in Eq. (17), the Fourier feature parameter $\mathbf{B} = [B_1, B_2, \dots, B_N] \in \mathbb{R}^N$ is initialized as a Gaussian distribution $\mathcal{N}(\mathbf{B}, \epsilon^2)$, and ϵ represents a constant small value. According to Eq. (18), the neural tangent kernel $\mathbf{K}$ of this simplified model can be given as:

$$\mathbf{K}_{ij} = \mathbf{K}(\mathbf{v}_i, \mathbf{v}_j) \approx N \cos(2\pi \mathbf{B}(\mathbf{v}_i - \mathbf{v}_j)). \quad (\text{C.1})$$

We consider the limit as the number of points to compute kernel $\mathbf{K}$ tend to infinity and follows a uniform distribution, then the eigenfunctions $g(\mathbf{v})$ with respect to the non-zero eigenvalue λ satisfy the following equation (Shawe-Taylor et al., 2005; Basri et al., 2020)

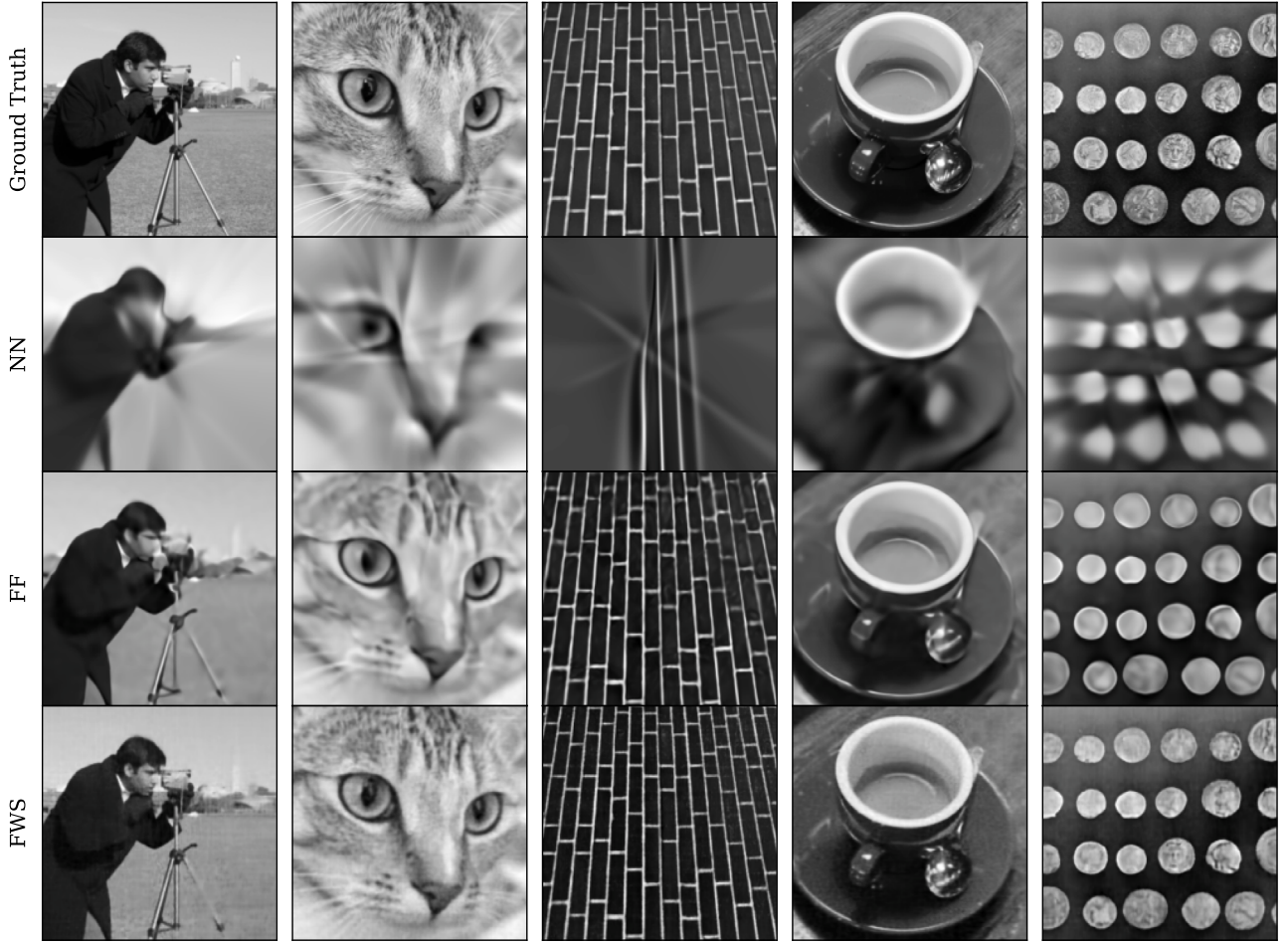


Fig. B.14. 2D image regression. Comparison of the predicted images generated by the fully-connected neural network (NN), the Fourier feature (FF) neural network with default setting, and with the Fourier Warm Start (FWS) method.

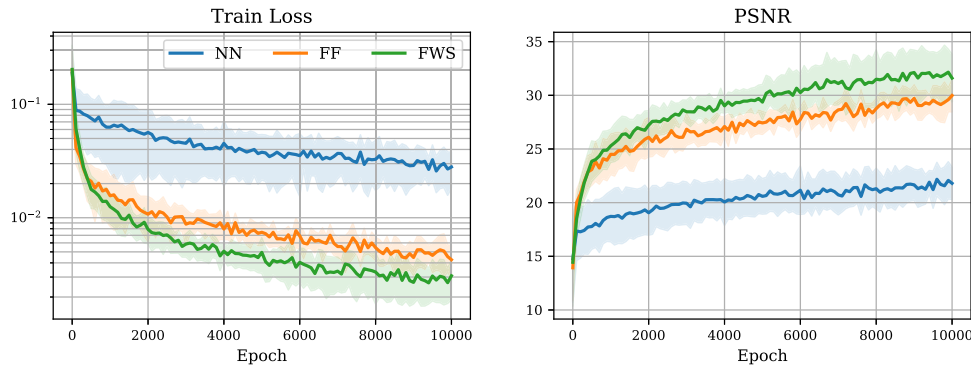


Fig. B.15. History of MSE loss and PSNR. Comparison of the training MSE loss and PSNR history for prediction by the fully-connected neural network (NN), the Fourier feature (FF) neural network with default setting, and with the Fourier Warm Start (FWS) method. The results are aggregated from 5 independent cases.

$$\int_{\mathbf{v}} \mathbf{K}(\mathbf{v}_i, \mathbf{v}_j) g(\mathbf{v}_j) d\mathbf{v}_j = \lambda g(\mathbf{v}_i). \quad (\text{C.2})$$

Then we have

$$N \int_{\mathbf{v}} \cos(2\pi B(\mathbf{v}_i - \mathbf{v}_j)) g(\mathbf{v}_j) d\mathbf{v}_j = \lambda g(\mathbf{v}_i), \quad (\text{C.3})$$

taking the second order derivative with respect to $\mathbf{v}_i$,

$$-4\pi^2 B^2 N \int_{\mathbf{v}} \cos(2\pi B(\mathbf{v}_i - \mathbf{v}_j)) g(\mathbf{v}_j) d\mathbf{v}_j = \lambda \frac{\partial^2 g(\mathbf{v}_i)}{\partial \mathbf{v}_i^2}. \quad (\text{C.4})$$

Substituting Eq. (C.3) into Eq. (C.4) gives

$$-4\pi^2 B^2 \lambda g(\mathbf{v}_i) = \lambda \frac{\partial^2 g(\mathbf{v}_i)}{\partial \mathbf{v}_i^2}. \quad (\text{C.5})$$

Therefore, it is clear that $g(\mathbf{v})$ satisfies

$$g''(\mathbf{v}) = -4\pi^2 B^2 g(\mathbf{v}). \quad (\text{C.6})$$

The general solution of this ordinary differential equation is

$$g(\mathbf{v}) = C_1 \cos(2\pi B\mathbf{v}) + C_2 \sin(2\pi B\mathbf{v}), \quad (\text{C.7})$$

where C_1 and C_2 are constants.

In order to get the corresponding eigenvalues, we substituted the expression of eigenfunction into Eq. (C.3). Then,

$$N \int_{\mathcal{V}} \cos(2\pi B(\mathbf{v}_i - \mathbf{v}_j)) [C_1 \cos(2\pi B\mathbf{v}_j) + C_2 \sin(2\pi B\mathbf{v}_j)] d\mathbf{v}_j = \lambda [C_1 \cos(2\pi B\mathbf{v}_i) + C_2 \sin(2\pi B\mathbf{v}_i)]. \quad (\text{C.8})$$

Next, the left-hand side of Eq. (C.8) can be written as,

$$\begin{aligned} LHS &= N \int_{\mathcal{V}} [\cos(2\pi B\mathbf{v}_i) \cos(2\pi B\mathbf{v}_j) + \sin(2\pi B\mathbf{v}_i) \sin(2\pi B\mathbf{v}_j)] \\ &\quad \cdot [C_1 \cos(2\pi B\mathbf{v}_j) + C_2 \sin(2\pi B\mathbf{v}_j)] d\mathbf{v}_j \\ &= N \cos(2\pi B\mathbf{v}_i) \int_{\mathcal{V}} \cos(2\pi B\mathbf{v}_j) [C_1 \cos(2\pi B\mathbf{v}_j) + C_2 \sin(2\pi B\mathbf{v}_j)] d\mathbf{v}_j \\ &\quad + N \sin(2\pi B\mathbf{v}_i) \int_{\mathcal{V}} \sin(2\pi B\mathbf{v}_j) [C_1 \cos(2\pi B\mathbf{v}_j) \\ &\quad + C_2 \sin(2\pi B\mathbf{v}_j)] d\mathbf{v}_j \\ &= N \cos(2\pi B\mathbf{v}_i) [C_1 M_1 + C_2 M_2] + N \sin(2\pi B\mathbf{v}_i) [C_1 M_2 + C_2 M_3], \end{aligned} \quad (\text{C.9})$$

notice that $\mathcal{V} = [0, 1]$, so we can get

$$\begin{aligned} M_1 &= \int_0^1 \cos(2\pi B\mathbf{v}_j) \cos(2\pi B\mathbf{v}_j) d\mathbf{v}_j = \frac{4\pi B + \sin(4\pi B)}{8\pi B}, \\ M_2 &= \int_0^1 \cos(2\pi B\mathbf{v}_j) \sin(2\pi B\mathbf{v}_j) d\mathbf{v}_j = \frac{1 - \cos(4\pi B)}{8\pi B}, \\ M_3 &= \int_0^1 \sin(2\pi B\mathbf{v}_j) \cos(2\pi B\mathbf{v}_j) d\mathbf{v}_j = \frac{4\pi B - \sin(4\pi B)}{8\pi B} \end{aligned} \quad (\text{C.10})$$

where

$$M_1 + M_3 = 1, \quad M_1 M_3 - M_2^2 = \frac{1}{4} - \frac{\sin^2(2\pi B)}{(4\pi B)^2} \quad (\text{C.11})$$

Then the Eq. (C.8) becomes

$$\begin{aligned} [C_1 M_1 + C_2 M_2] \cos(2\pi B\mathbf{v}_i) + [C_1 M_2 + C_2 M_3] \sin(2\pi B\mathbf{v}_i) \\ = \frac{\lambda C_1}{N} \cos(2\pi B\mathbf{v}_i) + \frac{\lambda C_2}{N} \sin(2\pi B\mathbf{v}_i), \end{aligned} \quad (\text{C.12})$$

Therefore,

$$\begin{aligned} C_1 M_1 + C_2 M_2 &= \frac{\lambda C_1}{N}, \\ C_1 M_2 + C_2 M_3 &= \frac{\lambda C_2}{N}. \end{aligned} \quad (\text{C.13})$$

The above equation can be simplified as

$$Av = \frac{\lambda}{N} v, \quad (\text{C.14})$$

where

$$A = \begin{bmatrix} M_1 & M_2 \\ M_2 & M_3 \end{bmatrix}, \quad v = \begin{bmatrix} C_1 \\ C_2 \end{bmatrix}. \quad (\text{C.15})$$

The characteristic polynomial of Eq. (C.14) can be written as

$$\begin{aligned} \det\left(\frac{\lambda}{N} I - A\right) &= \left(\frac{\lambda}{N} - M_1\right)\left(\frac{\lambda}{N} - M_3\right) - M_2^2 \\ &= \left(\frac{\lambda}{N}\right)^2 - (M_1 + M_3)\frac{\lambda}{N} + M_1 M_3 - M_2^2 \\ &= 0, \end{aligned} \quad (\text{C.16})$$

substituting the Eq. (C.11), then,

$$\left(\frac{\lambda}{N}\right)^2 - \frac{\lambda}{N} + \frac{1}{4} - \frac{\sin^2(2\pi B)}{(4\pi B)^2} = 0. \quad (\text{C.17})$$

We can get the eigenvalues as

$$\lambda = N\left(\frac{1}{2} \pm \frac{\sin(2\pi B)}{4\pi B}\right). \quad (\text{C.18})$$

This concludes the proof of Eqs. (19) and (20). $\square$

For a random Fourier feature (RFF) neural network, the proof process is completely identical.

Sinusoidal Feature (SF) neural network. Next, we consider the variants of the Fourier feature neural network, we still focus on a two-layer bias-free model and keep the initialization settings.

Proof. For SF neural network, the Eq. (18) can be rewritten as:

$$\begin{aligned} K_{ij} &= K(\mathbf{v}_i, \mathbf{v}_j) = \sum_{k=1}^N \sin(2\pi B_k \mathbf{v}_i) \sin(2\pi B_k \mathbf{v}_j) \\ &= \frac{1}{2} \sum_{k=1}^N [\cos(2\pi B_k (\mathbf{v}_i - \mathbf{v}_j)) - \cos(2\pi B_k (\mathbf{v}_i + \mathbf{v}_j))] \\ &\approx \frac{N}{2} [\cos(2\pi B (\mathbf{v}_i - \mathbf{v}_j)) - \cos(2\pi B (\mathbf{v}_i + \mathbf{v}_j))]. \end{aligned} \quad (\text{C.19})$$

Then substituting into the Eq. (C.2),

$$\frac{N}{2} \int_{\mathcal{V}} [\cos(2\pi B (\mathbf{v}_i - \mathbf{v}_j)) - \cos(2\pi B (\mathbf{v}_i + \mathbf{v}_j))] g(\mathbf{v}_j) d\mathbf{v}_j = \lambda g(\mathbf{v}_i), \quad (\text{C.20})$$

taking the second-order derivative concerning $\mathbf{v}_i$ results in

$$-2\pi^2 B^2 N \int_{\mathcal{V}} [\cos(2\pi B (\mathbf{v}_i - \mathbf{v}_j)) - \cos(2\pi B (\mathbf{v}_i + \mathbf{v}_j))] g(\mathbf{v}_j) d\mathbf{v}_j = \lambda \frac{\partial^2 g(\mathbf{v}_i)}{\partial \mathbf{v}_i^2}, \quad (\text{C.21})$$

then combined with Eq. (C.20) we can get

$$-4\pi^2 B^2 \lambda g(\mathbf{v}_i) = \lambda \frac{\partial^2 g(\mathbf{v}_i)}{\partial \mathbf{v}_i^2}. \quad (\text{C.22})$$

This is identical to Eq. (C.5), so we can achieve the same results for eigenvalues and eigenfunctions as

$$\lambda = N\left(\frac{1}{2} \pm \frac{\sin(2\pi B)}{4\pi B}\right). \quad (\text{C.23})$$

$$g(\mathbf{v}) = C_1 \cos(2\pi B\mathbf{v}) + C_2 \sin(2\pi B\mathbf{v}), \quad (\text{C.24})$$

where C_1 and C_2 are constants. $\square$

Cosinusoidal Feature (CF) neural network. Finally, we focus on a two-layer bias-free CF neural network with the same initialization strategy in Section 3.1.

Proof. We rewrite Eq. (18) as:

$$\begin{aligned} K_{ij} &= K(\mathbf{v}_i, \mathbf{v}_j) = \sum_{k=1}^N \cos(2\pi B_k \mathbf{v}_i) \cos(2\pi B_k \mathbf{v}_j) \\ &= \frac{1}{2} \sum_{k=1}^N [\cos(2\pi B_k (\mathbf{v}_i - \mathbf{v}_j)) + \cos(2\pi B_k (\mathbf{v}_i + \mathbf{v}_j))] \\ &\approx \frac{N}{2} [\cos(2\pi B (\mathbf{v}_i - \mathbf{v}_j)) + \cos(2\pi B (\mathbf{v}_i + \mathbf{v}_j))]. \end{aligned} \quad (\text{C.25})$$

Combined with the Eq. (C.2) we can get

$$\frac{N}{2} \int_{\mathcal{V}} [\cos(2\pi B (\mathbf{v}_i - \mathbf{v}_j)) + \cos(2\pi B (\mathbf{v}_i + \mathbf{v}_j))] g(\mathbf{v}_j) d\mathbf{v}_j = \lambda g(\mathbf{v}_i), \quad (\text{C.26})$$

likewise, taking the second-order derivative concerning $\mathbf{v}_i$ results in

$$-2\pi^2 B^2 N \int_{\mathcal{V}} [\cos(2\pi B (\mathbf{v}_i - \mathbf{v}_j)) + \cos(2\pi B (\mathbf{v}_i + \mathbf{v}_j))] g(\mathbf{v}_j) d\mathbf{v}_j = \lambda \frac{\partial^2 g(\mathbf{v}_i)}{\partial \mathbf{v}_i^2}, \quad (\text{C.27})$$

then combined with Eq. (C.26) we can get

$$-4\pi^2 B^2 \lambda g(\mathbf{v}_i) = \lambda \frac{\partial^2 g(\mathbf{v}_i)}{\partial \mathbf{v}_i^2}. \quad (\text{C.28})$$

The result is the same with Eq. (C.5), so we can achieve the eigenvalues and eigenfunctions as

$$\lambda = N\left(\frac{1}{2} \pm \frac{\sin(2\pi B)}{4\pi B}\right). \quad (\text{C.29})$$

$$g(\mathbf{v}) = C_1 \cos(2\pi B\mathbf{v}) + C_2 \sin(2\pi B\mathbf{v}), \quad (\text{C.30})$$

where C_1 and C_2 are constants. $\square$

References

- Basri, R., Galun, M., Geifman, A., Jacobs, D., Kasten, Y., Kritchman, S., 2020. Frequency bias in neural networks for input of non-uniform density. In: III, H.D., Singh, A. (Eds.), Proceedings of the 37th International Conference on Machine Learning. In: Proceedings of Machine Learning Research, vol. 119, PMLR, pp. 685–694, URL: <https://proceedings.mlr.press/v119/basri20a.html>.
- Baydin, A.G., Pearlmutter, B.A., Radul, A.A., Siskind, J.M., 2018. Automatic differentiation in machine learning: a survey. *J. Mach. Learn. Res.* 18, 1–43.
- Bottou, L., Curtis, F.E., Nocedal, J., 2018. Optimization methods for large-scale machine learning. *SIAM Rev.* 60 (2), 223–311.
- Cai, W., Li, X., Liu, L., 2020. A phase shift deep neural network for high frequency approximation and wave problems. *SIAM J. Sci. Comput.* 42 (5), A3285–A3312.
- Cai, S., Mao, Z., Wang, Z., Yin, M., Karniadakis, G.E., 2021. Physics-informed neural networks (PINNs) for fluid mechanics: A review. *Acta Mech. Sin.* 37 (12), 1727–1738.
- Cao, Y., Fang, Z., Wu, Y., Zhou, D.-X., Gu, Q., 2021. Towards understanding the spectral bias of deep learning. In: Zhou, Z.-H. (Ed.), Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, IJCAI-21. International Joint Conferences on Artificial Intelligence Organization, pp. 2205–2211, URL: <https://doi.org/10.24963/ijcai.2021/304>. Main Track.
- Cuomo, S., Di Cola, V.S., Giampaolo, F., Rozza, G., Raissi, M., Piccialli, F., 2022. Scientific machine learning through physics-informed neural networks: where we are and what's next. *J. Sci. Comput.* 92 (3), 88.
- Geifman, A., Galun, M., Jacobs, D., Ronen, B., 2022. On the spectral bias of convolutional neural tangent and gaussian process kernels. *Adv. Neural Inf. Process. Syst.* 35, 11253–11265.
- Glorot, X., Bengio, Y., 2010. Understanding the difficulty of training deep feedforward neural networks. In: Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics. JMLR Workshop and Conference Proceedings, pp. 249–256.
- Jacot, A., Gabriel, F., Hongler, C., 2018. Neural tangent kernel: Convergence and generalization in neural networks. *Adv. Neural Inf. Process. Syst.* 31.
- Jagtap, A.D., Kharazmi, E., Karniadakis, G.E., 2020. Conservative physics-informed neural networks on discrete domains for conservation laws: Applications to forward and inverse problems. *Comput. Methods Appl. Mech. Engrg.* 365, 113028.
- Kharazmi, E., Zhang, Z., Karniadakis, G.E., 2021. hp-VPINNs: Variational physics-informed neural networks with domain decomposition. *Comput. Methods Appl. Mech. Engrg.* 374, 113547.
- Kingma, D.P., Ba, J., 2015. Adam: A method for stochastic optimization. In: Bengio, Y., LeCun, Y. (Eds.), 3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7–9, 2015, Conference Track Proceedings. URL: <http://arxiv.org/abs/1412.6980>.
- Krishnapriyan, A., Gholami, A., Zhe, S., Kirby, R., Mahoney, M.W., 2021. Characterizing possible failure modes in physics-informed neural networks. In: Ranzato, M., Beygelzimer, A., Dauphin, Y., Liang, P., Vaughan, J.W. (Eds.), Advances in Neural Information Processing Systems. Curran Associates, Inc., pp. 26548–26560.
- Lee, J., Xiao, L., Schoenholz, S., Bahri, Y., Novak, R., Sohl-Dickstein, J., Pennington, J., 2019. Wide neural networks of any depth evolve as linear models under gradient descent. *Adv. Neural Inf. Process. Syst.* 32.
- Li, K., Tang, K., Wu, T., Liao, Q., 2019. D3M: A deep domain decomposition method for partial differential equations. *IEEE Access* 8, 5283–5294.
- Li, S., Xia, Y., Liu, Y., Liao, Q., 2023. A deep domain decomposition method based on Fourier features. *J. Comput. Appl. Math.* 423, 114963.
- Liu, Z., Cai, W., John Xu, Z.-Q., 2020. Multi-scale deep neural network (MscaledNN) for solving Poisson-Boltzmann equation in complex domains. *Commun. Comput. Phys.* 28 (5), 1970–2001, URL: http://global-sci.org/intro/article_detail/cicp/18402.html.
- Lu, L., Pestourie, R., Yao, W., Wang, Z., Verdugo, F., Johnson, S.G., 2021. Physics-informed neural networks with hard constraints for inverse design. *SIAM J. Sci. Comput.* 43 (6), B1105–B1132.
- Novak, R., Xiao, L., Hron, J., Lee, J., Alemi, A.A., Sohl-Dickstein, J., Schoenholz, S.S., 2020. Neural tangents: Fast and easy infinite neural networks in python. In: International Conference on Learning Representations. URL: <https://openreview.net/forum?id=SkID9yrFPS>.
- Orfanidis, S.J., 1995. Introduction to Signal Processing. Prentice-Hall, Inc..
- Rahaman, N., Baratin, A., Arpit, D., Draxler, F., Lin, M., Hamprecht, F., Bengio, Y., Courville, A., 2019. On the spectral bias of neural networks. In: International Conference on Machine Learning. PMLR, pp. 5301–5310.
- Rahimi, A., Recht, B., 2007a. Random features for large-scale kernel machines. *Adv. Neural Inf. Process. Syst.* 20.
- Rahimi, A., Recht, B., 2007b. Random features for large-scale kernel machines. In: Platt, J., Koller, D., Singer, Y., Roweis, S. (Eds.), Advances in Neural Information Processing Systems. Curran Associates, Inc., URL: <https://proceedings.neurips.cc/paper/2007/file/013a006f03dbc5392effeb8f18fda755-Paper.pdf>.
- Raissi, M., Perdikaris, P., Karniadakis, G.E., 2019. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *J. Comput. Phys.* 378, 686–707.
- Raissi, M., Yazdani, A., Karniadakis, G.E., 2020. Hidden fluid mechanics: Learning velocity and pressure fields from flow visualizations. *Science* 367 (6481), 1026–1030.
- Rao, K.R., Kim, D.N., Hwang, J.J., 2010. Fast Fourier Transform: Algorithms and Applications, vol. 32, Springer.
- Ronen, B., Jacobs, D., Kasten, Y., Kritchman, S., 2019. The convergence rate of neural networks for learned functions of different frequencies. *Adv. Neural Inf. Process. Syst.* 32.
- Sallam, O., Fürth, M., 2023. On the use of Fourier Features-Physics Informed Neural Networks (FF-PINN) for forward and inverse fluid mechanics problems. *Proc. Inst. Mech. Eng. M* 14750902231166424.
- Saputro, D.R.S., Widyansih, P., 2017. Limited memory Broyden-Fletcher-Goldfarb-Shanno (L-BFGS) method for the parameter estimation on geographically weighted ordinal logistic regression model (GWOLR). In: AIP Conference Proceedings. AIP Publishing LLC, 040009.
- Scarano, F., 2012. Tomographic PIV: principles and practice. *Meas. Sci. Technol.* 24 (1), 012001.
- Shawe-Taylor, J., Williams, C.K., Cristianini, N., Kandola, J., 2005. On the eigenspectrum of the Gram matrix and the generalization error of kernel-PCA. *IEEE Trans. Inform. Theory* 51 (7), 2510–2522.
- Smith, B.F., 1997. Domain Decomposition Methods for Partial Differential Equations. Springer.
- Sneddon, I.N., 1995. Fourier Transforms. Courier Corporation.
- Song, C., Wang, Y., 2023. Simulating seismic multifrequency wavefields with the Fourier feature physics-informed neural network. *Geophys. J. Int.* 232 (3), 1503–1514.
- Tancik, M., Srinivasan, P., Mildenhall, B., Fridovich-Keil, S., Raghavan, N., Singhal, U., Ramamoorthi, R., Barron, J., Ng, R., 2020a. Fourier features let networks learn high frequency functions in low dimensional domains. *Adv. Neural Inf. Process. Syst.* 33, 7537–7547.
- Tancik, M., Srinivasan, P., Mildenhall, B., Fridovich-Keil, S., Raghavan, N., Singhal, U., Ramamoorthi, R., Barron, J., Ng, R., 2020b. Fourier features let networks learn high frequency functions in low dimensional domains. In: Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., Lin, H. (Eds.), Advances in Neural Information Processing Systems. Curran Associates, Inc., pp. 7537–7547, URL: <https://proceedings.neurips.cc/paper/2020/file/55053683268957697aa39fa6f231c68-Paper.pdf>.
- Wang, S., Teng, Y., Perdikaris, P., 2021a. Understanding and mitigating gradient flow pathologies in physics-informed neural networks. *SIAM J. Sci. Comput.* 43 (5), A3055–A3081.
- Wang, S., Wang, H., Perdikaris, P., 2021b. On the eigenvector bias of Fourier feature networks: From regression to solving multi-scale PDEs with physics-informed neural networks. *Comput. Methods Appl. Mech. Engrg.* 384, 113938, URL: <https://www.sciencedirect.com/science/article/pii/S0045782521002759>.
- Wang, S., Yu, X., Perdikaris, P., 2022. When and why PINNs fail to train: A neural tangent kernel perspective. *J. Comput. Phys.* 449, 110768.
- Wang, B., Zhang, W., Cai, W., 2020. Multi-scale deep neural network (MscaledNN) methods for oscillatory Stokes flows in complex domains. *Commun. Comput. Phys.* 28 (5), 2139–2157, URL: http://global-sci.org/intro/article_detail/cicp/18407.html.
- Wong, J.C., Gupta, A., Ong, Y.-S., 2021. Can transfer neuroevolution tractably solve your differential equations? *IEEE Comput. Intell. Mag.* 16 (2), 14–30.
- Wong, J.C., Ooi, C., Gupta, A., Ong, Y.-S., 2022. Learning in sinusoidal spaces with physics-informed neural networks. *IEEE Trans. Artif. Intell.*
- Xu, Z.-Q.J., Zhang, Y., Xiao, Y., 2019. Training behavior of deep neural network in frequency domain. In: Neural Information Processing: 26th International Conference, ICONIP 2019, Sydney, NSW, Australia, December 12–15, 2019, Proceedings, Part I 26. Springer, pp. 264–274.
- Yao, Z., Gholami, A., Keutzer, K., Mahoney, M.W., 2020. Pyhessian: Neural networks through the lens of the hessian. In: 2020 IEEE International Conference on Big Data (Big Data). IEEE, pp. 581–590.